REVISITING CONSERVATIVENESS IN FLUID DYNAMICS: FAILURE OF NON-CONSERVATIVE PINNS AND A PATH-INTEGRAL REMEDY

Arun Govind Neelan
SimuNetics
Kanniyakumari
Tamil Nadu
India-629173
arunneelaniist@gmail.com

Ferdin Sagai Don Bosco
BosonQ Psi (BQP)
University of Pittsburgh
New York
USA-13202
ferdinsagai7@gmail.com

Naveen Sagar Jarugumalli
Airbus India
Bengaluru
Karnataka
India-560090
sagar.alex7@outlook.com

Suresh Balaji Vedarethinam
Indian Institute of Technology Kanpur
Uttar Pradesh
India-208016
vsbalaji15@gmail.com

April 2, 2026

ABSTRACT

The choice between conservative and non-conservative formulations is a fundamental dilemma in CFD. While non-conservative forms offer intuitive modeling in primitive variables, they typically produce erroneous shock speeds. This paper critically analyzes these formulations, contrasting classical failures against the capabilities of Physics-Informed Neural Networks (PINNs). Using the **Adaptive Weight and Viscosity (PINNs-AWV)** architecture, we evaluate cases ranging from shallow water equations to unsteady 1D and 2D Euler equations. Results reveal a significant dichotomy: while PINNs-AWV restores physical fidelity in scalar and steady systems, standard non-conservative PINNs fail in unsteady systems like the Sod shock tube. We demonstrate this failure stems from non-vanishing source terms introduced by viscous regularization, which violate the Rankine–Hugoniot jump conditions. To resolve this, we implement a **path-integral framework** based on Dal Maso–LeFloch–Murat (DLM) theory. By incorporating path-consistent losses in PINNs (**PI-PINN**) and using path-conservative numerical schemes, we successfully recover correct shock speeds within non-conservative frameworks. Our results prove the path-integral approach provides a rigorous mathematical bridge for physical accuracy in both classical and machine learning solvers, enabling primitive-variable formulations in transient, high-speed simulations.

***K*eywords** Physics Informed Deep Learning · Numerical Analysis · Supersonic Flows · Conservative scheme · Non-conservative schemes

1 Introduction

The conservative and non-conservative forms are two ways of writing the mathematical model of a system as a partial differential equation (PDE). The conservative form of a governing equation expresses the conservation law in divergence (flux) form. Mathematically, it is written as:

$$\frac{\partial \mathbf{U}}{\partial t} + \nabla \cdot \mathbf{F}(\mathbf{U}) = 0 \tag{1}$$

where, $\mathbf{U}$ vector of conserved variables, $\mathbf{F}$ is flux vector (which may depends on $\mathbf{U}$ for non-linear systems). Integrating (1) over the cell of x = [i-0.5, i+0.5], we get conservative discretization

$$\bar{\mathbf{U}}_i^{n+1} = \bar{\mathbf{U}}_i^n - \frac{\Delta t}{\Delta x_i}(\mathbf{F}_{i+1/2}^n - \mathbf{F}_{i-1/2}^n), \tag{2}$$

where, i is the cell center, and $i + 1/2$ and $i - 1/2$ are the cell faces. Equation 2 is called conservative discretization. The numerical schemes based on the conservative formulations include the finite volume method [1] and the conservative finite difference method [2]. In this work, reference to "conservative equations" will mean equations that can be written in the form of (1). Non-conservative form can be formulated by applying the chain rule to the conservative equation expressed in (1). For illustration, consider the law of mass conservation that is expressed in its conservative form as shown in eq. 3. Here, the flux of interest is $\rho\mathbf{u}$.

$$\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{u}) = 0 \tag{3}$$

On applying the product rule to the flux term, the resulting PDE can be expressed as, shown in eq. 4 The non-conservative form of the continuity equation is

$$\frac{D\rho}{Dt} + \rho(\nabla \cdot \mathbf{u}) = 0, \tag{4}$$

where, $\frac{D\rho}{Dt}$ is the total/substantial derivative expressed as, $\frac{D\rho}{Dt} = \frac{\partial \rho}{\partial t} + \mathbf{u}.\nabla$, with $\mathbf{u}.\nabla\rho$ representing the convective derivative. Mathematically, the conservative and non-conservative form are equivalent, with each being derivable from the other without any additional "mathematical" assumptions.

These restrictions on the non-conservative formulation make it necessary to account for the physics of the system while selecting the form of the governing PDE. For instance, compressible flows with shocks is capable of accurately resolving the fluxes across the shock if the conservative form is utilized [3]. On the other hand, incompressible or smooth-flow regimes, where conservation across discontinuities is less critical, are more amenable to the non-conservation form of the equation. Additionally, this form operates directly on the primitive variables—velocity, pressure, and density—making it more intuitive than solutions from the conservative form, wherein the primitive variables need to be calculated from the fluxes.

Application of the non-conservative form to compressible flows can be erroneous because this form lacks the ability to rigorously preserve conserved quantities across control volumes, leading to inaccurate solutions near shocks or discontinuities. Alternatively, conservative schemes tend to exhibit slower convergence for low Mach number flows. This is primarily because one of the eigenvalues of the flux Jacobian matrix—associated with the acoustic waves—approaches zero as the Mach number decreases [4]. Consequently, the system becomes stiff, leading to ill-conditioning in the numerical flux evaluations and degrading convergence performance. This can be overcome using pre-conditioners [5], [6].

Rankine-Hugoniot relations ensure correct transport of mass, momentum, and energy across the discontinuity. The non-conservative form does not inherently satisfy these conditions. However, this comes at the cost of violating conservation laws, especially when strong gradients or shocks develop. It's important to reiterate that while non-conservative forms can converge faster for smooth, shock-free flows, their fundamental inability to correctly capture discontinuities without special treatment makes the conservative form the default choice for general-purpose Computational Fluid Dynamics (CFD) packages. Though conservative schemes are widely accepted in CFD simulation packages, their implementation can sometimes reduce accuracy or efficiency on low-speed flows and the Schrodinger equation [7]. The "faster convergence" of the non-conservative form is a niche advantage, primarily relevant in specific applications where solution smoothness is guaranteed or shock-fitting techniques are employed. The choice between conservative and non-conservative schemes should be based on the problem's nature, the importance of conservation, and the desired balance between accuracy, stability, and computational cost. The summary of the advantages and disadvantages is tabulated in Table 1.

While conservative formulations are ideal for problems involving shocks due to their ability to preserve physical quantities across discontinuities, there are many important cases where it is not possible to express the governing equations in a purely conservative form. A classic example is the shallow water equations (SWE): while they admit a conservative form over a flat bottom, introducing variable bottom topography breaks the conservative structure [8], [9]. Several other systems inherently involve non-conservative terms. For instance, the *Baer–Nunziato model* for compressible two-phase flows includes non-conservative interfacial interaction terms, which are essential for modeling phase exchange dynamics [10].

Table 1: Comparison of conservative and non-conservative forms of governing equations

Aspect	**Conservative Form**	**Non-Conservative Form**
High speed flows	Accurate shock prediction	may give wrong shocks
Low speed flows	Slow convergence	Ideal
Discretization	FVM/FDM	FDM
Computational cost	Slightly higher	Lower
Conservation Property	Built-in	Mesh-dependent
Suitability	Compressible/Discontinuous flows	Smooth/Incompressible flows

In *nonlinear solid mechanics*, especially for materials exhibiting history-dependent or path-dependent behavior, the governing equations frequently appear in non-conservative form [11]. Although the ideal magneto-hydrodynamics (MHD) equations can be expressed conservatively, the divergence-free condition on the magnetic field ($\nabla \cdot \mathbf{B} = 0$) poses significant numerical challenges. Many schemes introduce non-conservative correction terms to maintain this constraint [12]. Similarly, *advanced traffic flow models*, particularly those accounting for non-local interactions or anticipatory driver behavior, incorporate non-conservative terms to accurately capture emergent dynamics [13]. The *Richards equation*, which models unsaturated groundwater flow in porous media, combines mass conservation with a non-linear form of Darcy's law and cannot be written in conservative form [14]. In the case of *non-Newtonian fluids*, the shear stress terms in the momentum equations are often non-conservative, especially when complex rheological models are involved [15]. Similarly, the *Coriolis force* arising in rotating systems generally cannot be cast in a conservative form [16].

To ensure that non-conservative numerical schemes predict the correct shock speed, several strategies have been developed, each addressing different aspects of shock dynamics and entropy consistency. One widely used approach is the class of *path-conservative methods* [17], [18], which generalizes the notion of fluxes by integrating along paths in state space. These methods ensure a consistent treatment of non-conservative products while preserving key conservation properties of the system. Another effective strategy involves introducing artificial viscosity terms in a manner that respects the entropy structure of the continuous equations. When combined with suitable entropy-stable discretizations, this approach helps enforce the correct selection of physically admissible shock solutions [19]. Augmented Riemann solvers provide yet another powerful tool: by explicitly incorporating non-conservative terms into the wave decomposition, they improve the resolution of discontinuities and shock speeds [20]. Similarly, *well-balanced schemes* are designed to exactly preserve steady-state solutions, thereby reducing spurious oscillations near shocks and aiding in the accurate prediction of shock propagation [21].

Additional techniques to improve the accuracy of numerical schemes include *entropy fixes* and *flux modifications*, which are designed to enforce entropy conditions in discrete settings [22]. Another important approach is *adaptive mesh refinement (AMR)*, which locally enhances resolution near shocks, thereby improving the accuracy of non-conservative schemes in complex flow regimes [23]. Collectively, these strategies help non-conservative formulations achieve reliable shock predictions while addressing the challenges posed by discontinuities and entropy violations. Furthermore, spurious oscillations can be partially suppressed by applying a non-conservative correction to the total energy through a coupled evolution equation for pressure, as demonstrated in [24].

The procedures mentioned above allow non-conservative schemes to handle problems involving shocks and discontinuities. Although their implementation is generally straightforward, stabilizing the solver is slightly difficult because of a lack of a popular solver to handle a variety of flows. Among these, artificial viscosity-based entropy fix methods are relatively straightforward to implement; however, they require extensive tuning to determine the appropriate artificial viscosity coefficient for a given problem. Most existing solvers rely on suboptimal artificial viscosity values. These algorithms can be fine-tuned using deep learning techniques by training on multiple forward problems [25], [26].

A more direct and effective approach is to use Physics-Informed Neural Networks (PINNs) with adaptive viscosity [27], [28], where the PINNs framework automatically learns the optimal viscosity required to stabilize the solution. In this work, we adopt this strategy and investigate whether it can effectively handle shocks in both conservative and non-conservative formulations of partial differential equations. Specifically, we focus on the Burgers' equation, and both steady and unsteady Euler equations.

The shock speed estimation is a critical step in high-speed solvers, which is estimated using the Rankine-Hugoniot (RH) jump condition. The RH condition for the conservative equation is:

$$s = \frac{f(u_R) - f(u_L)}{u_R - u_L} \tag{5}$$

Equation (5) is derived and defined for the conservative form, so the conservative form is commonly used in computational fluid dynamics. When we use the non-conservative form on a problem having shocks and solve it using numerical methods, it gives the wrong shock speed.

2 What is Conservation?

Conservation refers to the preservation of a physical quantity under system evolution. In continuum mechanics, conservation laws arise from integral balance principles [9]:

$$\frac{d}{dt}\int_{\Omega} U\, d\Omega = -\int_{\partial\Omega} \mathbf{F}(U)\cdot \mathbf{n}\, dS + \int_{\Omega} S\, d\Omega, \tag{6}$$

which states that the rate of change of a quantity within a control volume equals the net flux across its boundary plus volumetric sources. Using the divergence theorem, this leads to the local conservative form:

$$\frac{\partial U}{\partial t} + \nabla\cdot \mathbf{F}(U) = S. \tag{7}$$

While this formulation expresses conservation at the continuous level, the notion of conservativeness extends beyond the governing equations and must also be considered at numerical and physical levels [3].

2.1 Types of Conservativeness

2.1.1 Mathematical Conservativeness

A system is mathematically conservative if it can be expressed in divergence (flux) form as shown in Eq. (7). This ensures:

- Consistency with integral balance laws,
- Existence of weak solutions [29],
- Correct discontinuity propagation through Rankine–Hugoniot conditions.

Mathematical conservativeness is a property of the governing equations. However, it does not guarantee that a numerical approximation will converge to the correct weak solution.

2.1.2 Numerical Conservativeness

A numerical method is conservative if it preserves the discrete integral balance:

$$\frac{d}{dt}\sum_{i} U_i \Delta V_i = \text{boundary fluxes} + \text{discrete sources}. \tag{8}$$

This is typically achieved through flux-difference formulations such as finite volume methods, where interface fluxes are shared between adjacent cells. As noted by [9], numerical conservativeness ensures:

- Exact conservation at the discrete level (up to machine precision),
- Correct shock speeds (Telescoping property),
- Convergence toward admissible weak solutions (Lax-Wendroff Theorem).

Without numerical conservation, even stable and consistent schemes may yield incorrect discontinuity locations and accumulated global errors.

2.1.3 Physical Conservativeness

Physical conservativeness refers to the preservation of fundamental physical principles beyond simple mass/momentum balance. This includes:

- **Positivity:** Ensuring density ($\rho > 0$), pressure, or water depth remain non-negative.
- **Entropy consistency:** Ensuring the solution satisfies the Second Law of Thermodynamics to avoid non-physical expansion shocks [30].
- **Well-balanced property:** The preservation of equilibrium states (e.g., hydrostatic balance in shallow water equations).

A scheme may satisfy mathematical and numerical conservation while violating physical constraints, leading to spurious oscillations or simulation crashes.

2.1.4 Why Conservativeness is Necessary

Conservativeness is required for three primary reasons:

1. **Correct discontinuity propagation:** Non-conservative discretizations may converge to incorrect shock speeds [31].
2. **Long-time stability:** Small conservation errors accumulate and cause drift in atmospheric and climate simulations.
3. **Physical credibility:** Violation of physical constraints results in nonphysical states and unreliable predictions.

2.1.5 Hierarchy of Conservativeness

The three notions form a hierarchy:

$$\text{Physical} \supset \text{Numerical} \supset \text{Mathematical}. \tag{9}$$

Mathematical conservativeness defines the correct continuous model, numerical conservativeness ensures correct discrete representation, and physical conservativeness guarantees conservation and long-term stability. Table 2 shows the hierarchy of conservativeness in computational modeling. There are few problems where we no need to worry about these hierarchy when solving but some problems different levels of considerations is required. We will study those in upcoming sections.

Table 2: Hierarchy of conservativeness in computational modeling.

Level	What is Conserved	Enforced By	Failure Leads To
Mathematical	PDE invariants	Divergence form	Incorrect weak solution
Numerical	Discrete integrals	Flux-consistent schemes	Wrong shock speed
Physical	Fundamental principles	Constraints	Instability

2.2 Illustration: Non-conservative form leads to wrong shock speed

We shall try to understand it through one example. Consider the inviscid Burgers' equation:

$$\frac{\partial u}{\partial t} + \frac{\partial}{\partial x}\left(\frac{u^2}{2}\right) = 0, \tag{10}$$

which is a scalar conservation law with flux function:

$$f(u) = \frac{u^2}{2}. \tag{11}$$

Lets assume we have following Riemann initial condition

$$u_L = 2 \quad \text{(left state)}, \qquad u_R = 0 \quad \text{(right state)}.$$

The Rankine–Hugoniot (RH) condition gives the shock speed s as:

$$s = \frac{f(u_R) - f(u_L)}{u_R - u_L} = \frac{\frac{u_R^2}{2} - \frac{u_L^2}{2}}{u_R - u_L} = \frac{1}{2}(u_R + u_L). \tag{12}$$

Substituting the values:

$$s = \frac{1}{2}(0+2) = 1. \tag{13}$$

Therefore, the shock propagates to the right with speed $s = 1$. In contrast, if one writes the Burgers' equation in non-conservative form:

$$\frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} = 0, \tag{14}$$

The expression becomes ill-defined at the discontinuity due to the ambiguity in interpreting the product $u\frac{\partial u}{\partial x}$. Thus, the non-conservative form fails to correctly capture the shock speed.

2.3 Illustration: The term in the conservative equation should be physically conserved for correct wave speed

Another requirement to get the accurate shock speed is that the quantity in the conservative form of PDE should be physically conservative; otherwise, it may lead to the wrong shock speed. This is the illustration based on the example given in Toros' books on *Riemann Solvers and Numerical Methods for Fluid Dynamics: A Practical Introduction* [3]. The 1-D shallow water equation is:

$$\begin{bmatrix} \phi \\ \phi u \end{bmatrix}_t = -\begin{bmatrix} \phi u \\ \phi u^2 + \frac{1}{2}\phi^2 \end{bmatrix}_x \tag{15}$$

$$\begin{bmatrix} \phi \\ u \end{bmatrix}_t = -\begin{bmatrix} \phi u \\ \frac{1}{2}u^2 + \phi \end{bmatrix}_x \tag{16}$$

Equation. (15) and (16) are mathematically the same and conservative but they lead to different shock speed. Both the equation are in the divergence form but they give different shock speed. That is because the conserved quantity present in the second row of (16) is not physically conservative.

Wave speed estimation using the second row of (15)**:** Using the Rankine-Hugoniot condition on the second row, we get

$$s\,[\phi u] = \left[\phi u^2 + \frac{1}{2}\phi^2\right], \tag{17}$$

where the jump $[\cdot]$ denotes the difference across the shock:

$$[\cdot] = (\cdot)_r - (\cdot)_l.$$

Expanding, this gives

$$s(\phi_r u_r - \phi_l u_l) = \left(\phi_r u_r^2 + \frac{1}{2}\phi_r^2\right) - \left(\phi_l u_l^2 + \frac{1}{2}\phi_l^2\right). \tag{18}$$

Solving for the shock speed s, we get

$$s = \frac{\phi_r u_r^2 + \frac{1}{2}\phi_r^2 - \phi_l u_l^2 - \frac{1}{2}\phi_l^2}{\phi_r u_r - \phi_l u_l}. \tag{19}$$

Let's try to estimate this using a numerical example. Given the left and right states

$$\phi_l = 2.0, \quad u_l = 3.0, \quad \phi_r = 1.0, \quad u_r = 1.0,$$

The wave speed is

$$s_1 = 3.7.$$

Wave speed estimation using the second row of (16)**:** Using the Rankine-Hugoniot condition on the second row, we have

$$s[u] = \left[\frac{1}{2}u^2 + \phi\right], \tag{20}$$

where the jump $[\cdot]$ denotes the difference across the shock:

$$[\cdot] = (\cdot)_r - (\cdot)_l.$$

Expanding, this gives

$$s(u_r - u_l) = \left(\frac{1}{2}u_r^2 + \phi_r\right) - \left(\frac{1}{2}u_l^2 + \phi_l\right). \tag{21}$$

Solving for the shock speed s, we get

$$s = \frac{\frac{1}{2}u_r^2 + \phi_r - \frac{1}{2}u_l^2 - \phi_l}{u_r - u_l}. \tag{22}$$

Both the form can give the same shock speed only if $\phi_r = \phi_l$. Let's try to estimate this using a numerical example. Given the left and right states

$$\phi_l = 2.0, \quad u_l = 3.0, \quad \phi_r = 1.0, \quad u_r = 1.0,$$

and hence

$$s_2 = 2.5.$$

Though (15) and (16) both are in divergence form and valid shallow water equations, they gave the same wave speed for the first equation but a different wave speed for the second equation. When we write the equation in divergence form, it is good to ensure that the conserved variable in the form is a physically conserved quantity to estimate the correct wave speed. In the above example, u is not a conserved variable, but $\phi \times u$ is a conserved variable, so (15) gave the correct wave speed. Please note that, so far, we haven't introduced any discretization. So this issue arose before we introduced the numerical discretization. In addition to this, both forms are mathematically equivalent. So the possible limitation is that the wave speed formula is applicable only to conservative discretization. We get $s(\mathbf{U}_R - \mathbf{U}_L) = \mathbf{F}(\mathbf{U}_R) - \mathbf{F}(\mathbf{U}_L)$ over the control volume only when we use conservative discretization.

3 Role of conservation on Discretization Framework

To clearly distinguish the role of mathematical and numerical conservativeness, we briefly describe the discretizations employed in this work.

3.1 Conservative Flux-Difference Formulation

For a conservation law written in divergence form,

$$\frac{\partial U}{\partial t} + \frac{\partial F(U)}{\partial x} = 0, \tag{23}$$

a finite-volume discretization yields

$$\frac{d}{dt}U_i = -\frac{1}{\Delta x}\left(F_{i+1/2} - F_{i-1/2}\right), \tag{24}$$

where $F_{i+1/2}$ denotes a consistent numerical flux shared between adjacent cells.

This formulation ensures exact discrete conservation:

$$\frac{d}{dt}\sum_i U_i \Delta x = F_{1/2} - F_{N+1/2}. \tag{25}$$

Such schemes converge to weak solutions satisfying the Rankine–Hugoniot condition and therefore produce correct shock speeds under mesh refinement.

3.2 Non-Conservative Formulation

Alternatively, the system may be written in quasi-linear form,

$$\frac{\partial U}{\partial t} + A(U)\frac{\partial U}{\partial x} = 0, \tag{26}$$

which does not explicitly express flux differences.

A standard discretization reads

$$\frac{d}{dt}U_i = -A(U_i)\frac{U_{i+1} - U_{i-1}}{2\Delta x}. \tag{27}$$

Although consistent and potentially stable, this formulation does not preserve a discrete flux balance. Consequently, it may converge to incorrect weak solutions and yield erroneous shock speeds. We will study the effect of this on PINNs and numerical methods in upcoming sections.

3.3 Path-Conservative and Viscosity-Corrected Form

To recover correct discontinuity propagation within a non-conservative framework, we incorporate either:

1. A path-integral treatment of non-conservative products, or
2. An adaptive artificial viscosity consistent with jump conditions.

In the path-conservative formulation, interface contributions are defined as

$$\int_0^1 A(\Phi(s))\frac{d\Phi}{ds}\,ds, \tag{28}$$

where $\Phi(s)$ is a path connecting left and right states. This construction restores consistency with generalized Rankine–Hugoniot conditions. Alternatively, artificial viscosity terms of the form

$$\frac{\partial}{\partial x}\left(\mu(U)\frac{\partial U}{\partial x}\right) \tag{29}$$

are introduced, where $\mu(U)$ is adaptively designed to activate near discontinuities.

3.4 Test Case 1: Mathematical Conservativeness via Burgers' Equation

To evaluate mathematical conservativeness and weak-solution behavior, we consider the inviscid Burgers' equation in strictly conservative form:

$$\frac{\partial u}{\partial t} + \frac{\partial}{\partial x}\left(\frac{u^2}{2}\right) = 0, \quad x \in [-1, 1]. \tag{30}$$

We prescribe a Riemann initial condition with $u_L = 1$ ($x < 0$) and $u_R = 0$ ($x \geq 0$). This generates a shock wave with a propagation speed determined by the Rankine–Hugoniot condition, $s = \Delta f/\Delta u = 0.5$, yielding the exact shock location $x_s(t) = 0.5t$. To demonstrate the impact of conservativeness, we compare two distinct discretizations:

1. Conservative Discretization We employ a standard first-order upwind scheme based on the flux formulation. The update formula preserves the cell averages:

$$u_i^{n+1} = u_i^n - \frac{\Delta t}{\Delta x}\left(F_{i+1/2} - F_{i-1/2}\right), \tag{31}$$

where the numerical flux is given by the Godunov flux for the scalar Burgers equation:

$$F_{i+1/2} = \max\left(f(u_i^n), f(u_{i+1}^n)\right) \quad \text{if } u_i > u_{i+1}. \tag{32}$$

This formulation ensures that $\sum u_i$ is conserved (telescoping sum property) and converges to the correct shock speed $s = 0.5$.

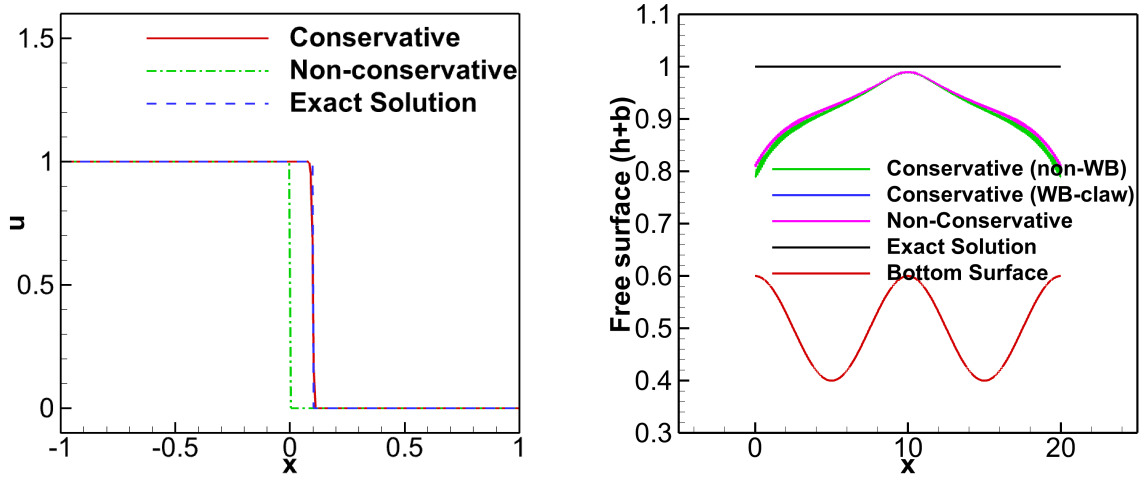
2. Non-Conservative Discretization Alternatively, we discretize the quasi-linear form $u_t + uu_x = 0$ using non-conservative upwinding:

$$u_i^{n+1} = u_i^n - \frac{\Delta t}{\Delta x}u_i^n\left(u_i^n - u_{i-1}^n\right), \quad \text{for } u_i > 0. \tag{33}$$

Although this scheme is consistent ($O(\Delta x)$ truncation error) and stable under CFL conditions, it does not satisfy the discrete integral balance. As shown by Hou and LeFloch (1994), this form converges to a non-physical weak solution with an incorrect shock speed of $s = 0$.

Numerical Setup

The domain is discretized using a uniform grid with $N_x = 400$. Time integration is performed using an explicit scheme with CFL $= 0.4$ up to $t_{\text{final}} = 0.2$ is shown in figure 1a. In this figure we can observe that non-conservative scheme unable to resolve the correct shock speed but conservative scheme able locate exact shock speed.



(a) Burgers equation

(b) Shallow water equation

Figure 1: Solution using conservative and non-conservative scheme

Role in Assessing Conservativeness

This test case isolates mathematical and numerical conservativeness as follows:

- The governing PDE is mathematically conservative by construction.
- A conservative flux-difference discretization preserves discrete integral balance and converges to the correct weak solution.
- A non-conservative discretization, despite consistency and stability, may violate discrete flux balance and yield incorrect shock speeds.

3.5 Test Case 2: Unified Assessment of Conservativeness Using the Shallow Water Equations

To simultaneously illustrate mathematical, numerical, and physical conservativeness, we consider the one-dimensional shallow water equations (SWE) with bottom topography:

$$\frac{\partial h}{\partial t} + \frac{\partial (hu)}{\partial x} = 0, \tag{34}$$

$$\frac{\partial (hu)}{\partial t} + \frac{\partial}{\partial x}\left(hu^2 + \frac{1}{2}gh^2\right) = -gh\frac{\partial b}{\partial x}. \tag{35}$$

Here, h denotes water depth, u velocity, $b(x)$ bottom elevation, and g gravitational acceleration.

Mathematical Conservativeness

The SWE are mathematically conservative in the variables (h, hu) when written in divergence form. The system satisfies integral balance laws and admits physically relevant steady solutions. In particular, the lake-at-rest equilibrium:

$$u = 0, \qquad h + b = \text{constant}, \tag{36}$$

represents a non-trivial steady state resulting from exact balance between flux gradients and the source term.

Computational Setup

The computational domain is

$$x \in [0, L], \quad L = 20,$$

discretized using $N_x = 400$ uniform grid points with spacing $\Delta x = L/(N_x - 1)$.

The bottom profile is prescribed as:

$$b(x) = b_0 + A_b \cos\left(\frac{2\pi x}{L_b}\right), \tag{37}$$

with parameters $b_0 = 0.5$, $A_b = 0.1$, and $L_b = 10$.

Initial conditions are:

$$h(x, 0) = H_0 - b(x) + A_p \exp\left(-\frac{(x - x_c)^2}{2\sigma^2}\right), \tag{38}$$

$$u(x, 0) = 0, \tag{39}$$

where $H_0 = 1.0$, $A_p = 0.0$, $x_c = L/2$, and $\sigma = 0.6$.

Time integration uses a second-order Runge–Kutta scheme with time step determined by the CFL condition: $\Delta t = \text{CFL}\frac{\Delta x}{\sqrt{gH_0}}, \quad \text{CFL} = 0.4$, and final time $T_{\text{end}} = 5$.

3.5.1 Discretizations Used in the SWE Test Case

To isolate the role of different levels of conservativeness, we employ multiple spatial discretizations of the shallow water equations.

1. Quasi-Linear (Non-Conservative) Finite Difference Formulation The governing equations are rewritten in primitive-variable form:

$$h_t = -uh_x - hu_x, \tag{40}$$

$$u_t = -uu_x - gh_x - gb_x. \tag{41}$$

Spatial derivatives are approximated using second-order centered finite differences:

$$q_x \approx \frac{q_{i+1} - q_{i-1}}{2\Delta x}. \tag{42}$$

Although this formulation is consistent and formally second-order accurate, it does not preserve a discrete flux balance in (h, hu). Consequently:

- Total mass is not preserved exactly,
- Total momentum exhibits drift,
- Equilibrium states are not maintained.

This discretization violates numerical conservativeness.

2. Conservative Finite-Volume Formulation The SWE are written in conservative vector form:

$$\frac{\partial \mathbf{U}}{\partial t} + \frac{\partial \mathbf{F}(\mathbf{U})}{\partial x} = \mathbf{S}(\mathbf{U}), \tag{43}$$

with

$$\mathbf{U} = \begin{pmatrix} h \\ hu \end{pmatrix}, \quad \mathbf{F}(\mathbf{U}) = \begin{pmatrix} hu \\ hu^2 + \frac{1}{2}gh^2 \end{pmatrix}, \quad \mathbf{S} = \begin{pmatrix} 0 \\ -ghb_x \end{pmatrix}.$$

A semi-discrete finite-volume formulation reads:

$$\frac{d}{dt}\mathbf{U}_i = -\frac{1}{\Delta x}\left(\mathbf{F}_{i+1/2} - \mathbf{F}_{i-1/2}\right) + \mathbf{S}_i. \tag{44}$$

This formulation guarantees exact discrete conservation of mass:

$$\frac{d}{dt}\sum_i h_i \Delta x = 0. \tag{45}$$

However, unless the source term is discretized consistently with the flux gradient, equilibrium states may still be destroyed.

3. Conservative but Non-Well-Balanced Form In many standard implementations, the source term is discretized independently of the flux term:

$$S_i = -gh_i b_x. \tag{46}$$

Although global conservation is preserved, the discrete flux gradient does not exactly balance the source term at steady state. As a result:

- Spurious motion develops,
- The lake-at-rest equilibrium is not preserved,
- Free-surface distortion appears.

This demonstrates that numerical conservativeness alone does not ensure physical conservativeness.

4. Well-Balanced Formulation A well-balanced scheme is constructed such that the discrete flux gradient and source term cancel exactly for the equilibrium state:

$$u = 0, \quad h + b = \text{constant}.$$

This typically requires hydrostatic reconstruction or modified flux evaluation ensuring that:

$$\left(\frac{1}{2}gh^2\right)_x + ghb_x = 0 \tag{47}$$

is satisfied discretely.

Only under this condition are:

- Mathematical conservativeness,
- Numerical conservativeness,
- Physical equilibrium preservation

simultaneously satisfied. Figure 1b shows the free surface elevation of conservative scheme with and without well balanced treatment, non-conservative scheme. From this it is clear that conservative schemes without well balanced enforcement unable to produce the correct free surface elevation.

Interpretation

This single test case therefore illustrates:

- The governing equations are mathematically conservative.
- Non-conservative discretizations violate numerical conservativeness.
- Conservative discretizations may still violate physical equilibrium.
- Well-balanced schemes are required to ensure physical conservativeness.

Conservativeness in computational fluid dynamics is inherently hierarchical. Mathematical conservation defines the governing equations through divergence-form partial differential equations, ensuring integral properties hold in the continuous limit. Numerical conservation translates this to the discrete level, enforcing a rigorous flux balance across cell interfaces to correctly capture wave speeds and jump conditions. However, for many systems, these are necessary but insufficient.

Furthermore, a carefully balanced discretization is crucial to preserve physically meaningful equilibrium states, such as the lake at rest condition in the shallow water equations. Without physics-aware schemes that also enforce shock-consistent entropy for compressible flows, numerical methods may formally conserve quantities but still produce non-physical solutions. Ultimately, ensuring physical fidelity requires targeted enforcement at all three levels of this hierarchy.

3.6 Can Non-Conservative Schemes Be Used for Shock Problems?

In general, conservative schemes are necessary for hyperbolic conservation laws containing shocks, since they satisfy the Rankine–Hugoniot jump conditions and converge to the physically correct weak solution [32]. The Lax–Wendroff theorem [33] establishes that any consistent, conservative scheme converges (if it converges) to a weak solution of the conservation law. Consequently, non-conservative discretizations are often considered unsuitable for shock-dominated flows.

Nevertheless, non-conservative formulations are not inherently invalid and can be advantageous in several situations. Many physical models are naturally expressed in primitive variables rather than conserved variables, including low-Mach-number flows, multiphase systems, turbulence closures, and relativistic hydrodynamics. Conservative formulations may also introduce numerical stiffness or loss of accuracy due to cancellation errors, particularly when kinetic energy dominates internal energy. Furthermore, certain systems (e.g., Baer-Nunziato-type multiphase models [10]) contain non-conservative products that cannot be written purely in divergence form. Non-conservative formulations may therefore offer improved conditioning and modeling flexibility, especially when coupled with physics-informed machine learning approaches. The primary challenge is that non-conservative schemes may converge to incorrect shock solutions unless additional mechanisms enforce the proper jump conditions.

Path-Conservative (Path-Integral) Approach A rigorous framework for non-conservative hyperbolic systems was developed by Dal Maso, LeFloch, and Murat [18] through the definition of weak solutions using path integrals in state space. The key idea is to replace the ill-defined non-conservative product across discontinuities with an integral taken along a path connecting the left and right states. Numerically, path-conservative schemes [34] compute interface contributions as

$$\int_0^1 \mathbf{A}(\mathbf{\Phi}(s))\, \frac{d\mathbf{\Phi}}{ds}\, ds, \tag{48}$$

where $\mathbf{\Phi}(s)$ defines a path in state space between the two states. This formulation generalizes conservative finite-volume methods and recovers the correct Rankine–Hugoniot conditions. Such schemes are widely used for multiphase and non-equilibrium flows, although the choice of path can influence the selected weak solution.

Artificial Viscosity Artificial viscosity [35] provides an alternative stabilization mechanism by introducing controlled dissipation near steep gradients. This regularizes discontinuities into thin layers and enforces entropy conditions. Modern artificial viscosity methods include adaptive viscosity based on shock sensors [36], matrix viscosity aligned with characteristic fields, and data-driven or learned viscosity models. While effective for stabilization, excessive dissipation may smear shocks and reduce accuracy.

Hybrid and Entropy-Stable Strategies We can combine non-conservative formulations with entropy-stable dissipation [37, 38], nonlinear limiters, path-consistent interface treatments, and adaptive viscosity mechanisms. These approaches aim to retain the flexibility of primitive-variable formulations while ensuring physically correct shock capturing. Non-conservative schemes can be applied to problems with shocks provided that additional mechanisms enforce correct discontinuity behavior. Path-conservative formulations offer a mathematically rigorous framework, whereas artificial viscosity supplies practical stabilization. Such methods are particularly relevant for complex multiphysics systems and emerging physics-informed computational approaches where conservative formulations may be restrictive.

3.7 Test Case 3: Conservative vs Non-Conservative Formulations for the Sod Shock Tube

To investigate the behavior of non-conservative formulations in the presence of shocks and the effect of parabolic regularization, we consider the one-dimensional Sod shock tube governed by the Euler equations. The computational

domain is $x \in [0, 2]$ with an initial discontinuity at $x_0 = 1$. The initial states are

$$(\rho, u, p) = \begin{cases} (1.0,\ 0,\ 1.0), & x < x_0, \\ (0.125,\ 0,\ 0.1), & x > x_0, \end{cases} \tag{49}$$

with ratio of specific heats $\gamma = 1.4$. The solution is evolved until $T = 0.5$ using two different numerical approaches.

Conservative Finite-Volume Method The Euler equations in conservative form,

$$\mathbf{U}_t + \mathbf{F}(\mathbf{U})_x = 0, \tag{50}$$

are discretized using a first-order finite-volume method with the Rusanov (local Lax–Friedrichs) flux:

$$\mathbf{F}_{i+\frac{1}{2}} = \frac{1}{2}\left(\mathbf{F}_L + \mathbf{F}_R\right) - \frac{1}{2}\alpha_{i+\frac{1}{2}}(\mathbf{U}_R - \mathbf{U}_L), \tag{51}$$

where $\alpha_{i+\frac{1}{2}} = \max(|u_L| + c_L, |u_R| + c_R)$ is the maximum signal speed. This conservative scheme satisfies the Rankine–Hugoniot jump conditions and produces the correct weak solution.

Non-Conservative Primitive-Variable Formulation For comparison, the Euler equations are also solved in primitive variables (ρ, u, p) using a non-conservative formulation augmented with artificial viscosity:

$$\rho_t + u\rho_x + \rho u_x = \nu \rho_{xx}, \tag{52}$$

$$u_t + u u_x + \frac{1}{\rho} p_x = \nu u_{xx}, \tag{53}$$

$$p_t + u p_x + \gamma p u_x = \nu p_{xx}. \tag{54}$$

Spatial derivatives are approximated using central finite differences. The viscosity coefficient is chosen adaptively as

$$\nu = \frac{1}{2} a_{\max} \Delta x, \tag{55}$$

where $a_{\max} = \max(|u| + c)$ is the maximum local wave speed. This diffusion term introduces parabolic regularization analogous to Rusanov-type dissipation. Figure 2 demonstrates that the non-conservative scheme with viscous dissipation (non-conservative-μ) fails to predict the correct shock speed. As time progresses, the shock position obtained from the non-conservative formulation increasingly deviates from the exact solution, while the numerical solution remains apparently stable. This behavior is noteworthy, as stability alone does not guarantee physical accuracy. Table 3 summarize the different level of conservation required for the different equations considered in this work.

Table 3: Comparison of conservation requirements across different physical systems.

System / Problem	Math. Cons.	Num. Cons.	Well-Balanced	Shock/Entropy
Burgers' (Inviscid)	Required	Required	N/A	Essential
Shallow Water (Static)	Required	Required	Essential	Optional
Euler Equations	Required	Required	Recommended	Essential

4 Introduction to PINNs

In this section, we have presented the details of the basic PINNs architecture and the adaptive weight and viscosity architecture used in this work.

4.1 Introduction

Modern CFD possesses a powerful toolkit with Finite Difference, Finite Element, Finite Volume, Spectral Element, and meshless approaches such as Smoothed Particle Hydrodynamics. However, there are still significant hurdles that are inherently difficult for CFD to tackle. For one, the incorporation of available empirical data is a rich source of physical information and is incorporated into CFD through correlations and models, and not directly. CFD also requires the governing system of equations to be complete, i.e., all boundary conditions must be known to a desired order of accuracy especially for atmospheric simulations. This is a severe limitation to the accuracy that CFD can extend to; for

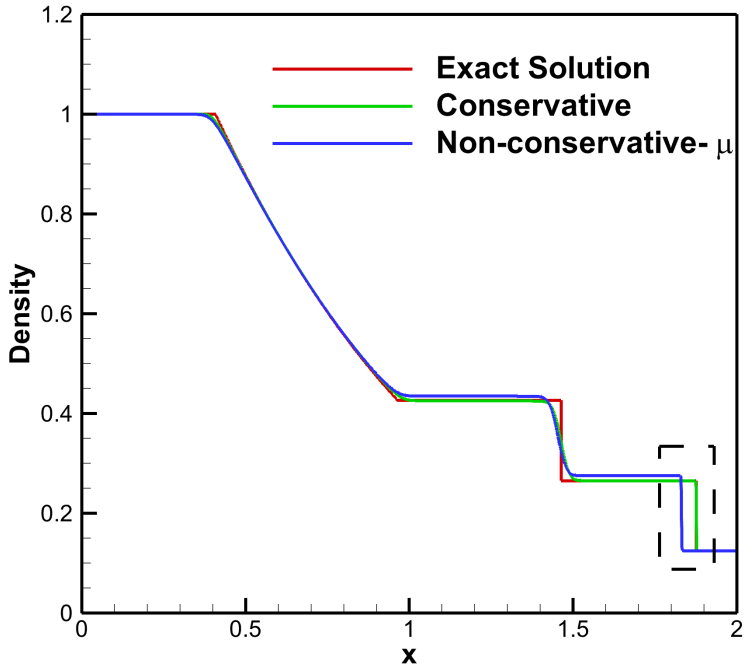


Figure 2: Density of Sod shock tube problem at T = 0.5 s

instance, available BC restricts CFD to the Navier-Stokes system, and advanced hydrodynamics cannot be captured through Burnett and super Burnett equations.

Statistical learning, a subfield of statistics focused on establishing a relationship between variables in a dataset and the output, has been modernized as Machine Learning (ML). The key principle of ML is to develop algorithms that learn from data to identify patterns, and make accurate predictions, with the added ability to continuously learn through experience. Rapid advancements in ML have revolutionized CFD research in many fields, offering a powerful data-driven approach. However, purely data-driven models lack the physical consistency and can lead to the learning process going astray. This has been observed by many researchers, and the unphysical results reported are an interesting counterpoint to the applicability of ML in CFD. Although well-trained models are great interpolators, they struggle when tasked with extrapolating beyond the training data.

The inspiration to include the rich theoretical knowledge into the data-driven ML process has resulted in a potent computational framework named Physics-Informed Neural Networks (PINNs). In this class of methods, the network's loss function, which consists of data loss, is augmented by a PDE loss, which signifies the deviation of the solution from the physical principles cast as a partial differential equation. In this paradigm, efforts to reduce the loss function is essentially efforts to ensure data and physical-fidelity of the solution.

The success of PINNs, specifically for solution to PDEs, can be attributed to their ability to leverage automatic differentiation (autograd or AD), which enables efficient and accurate computation of necessary derivatives to associate the inputs and outputs of the neural network. This feature enables PINNs to directly enforce the physics that are expressed through differential equations.

4.2 PINNs process

The entire process of setting up the PINNs for PDEs can be summarized by the following 3 steps,

- A neural network takes spatial and temporal coordinates as input and outputs the predicted solution to the physical system (e.g., temperature, velocity, displacement). Often, a fully connected feed-forward network is utilized for the network architecture.

- Automatic differentiation (AD) is a crucial component of PINNs. It allows for the efficient and accurate computation of the derivatives of the neural network's output with respect to its inputs. These derivatives are precisely what's needed to evaluate the terms in the differential equations for the physics loss.
- During training, an optimization algorithm (like Adam or L-BFGS) iteratively adjusts the neural network's parameters to minimize the total loss function (data loss + physics loss).

Developing a PINNs circuit from ground-up can be a daunting prospect from a coding standpoint. However, this is offset by the availability of standard frameworks such as PyTorch and TensorFlow, which have made the process extremely straightforward. However, despite the promise, PINNs face numerous challenges that need to be addressed. A large portion of these challenges is associated with enormous computational cost, while the next significant challenge arises due to generalizability. Computational cost can be tackled by decreasing the depth of the PINNs, which means there are fewer hidden layers and lesser number of trainable parameters. However, this severely restricts the expressive power of the PINNs. A complementary approach is to decompose the computational domain into multiple sub-domains and utilize a separate, low depth PINNs for each sub-domain. This approach lends itself to parallel computing naturally and reduces the cost associated with each PINNs ([39, 40]). Such Distributed PINNs (DPINNs) have been extensively explored in the context of CFD, resulting in several customized PINNs. Two examples of this philosophy are the Conservative PINNs (C-PINNs) [41] and the eXtended PINNs (XPINNs) [42], which vary in their decomposition strategies and problem types they can handle.

Conservative PINNs utilize a domain decomposition approach and are particularly well-suited to solve problems governed by strong conservation laws. Considering that the basis of CFD is the conservation of mass, momentum, and energy, these PINNs have shown a very relevant contribution to the present discussion. The critical challenge that is addressed by cPINNs is the enforcement of flux continuity across the interfaces of the decomposed domain, which is explicitly enforced through the strong form of the governing equations. Discrepancies in the enforcement are added to the loss functions and act as a penalty, which directs the solution towards a conservative solution. While cPINNs primarily focus on dividing the domain along its spatial dimensions and implementing shallow neural networks in each subdomain to learn the solution within these spatial sub-domains, XPINNs build upon and generalize cPINNs. XPINNs/Distributed PINNs expand the domain decomposition concept to both space and time. This allows them to handle a wider range of PDEs and employ arbitrary decomposition strategies.

Another approach to tackle the computational cost is to reduce the training time of the PINNs. This line of thinking has led to another variation of PINNs called the Variational PINNs (V-PINNs) [43]. Unlike the PINNs discussed above, VPINNs deviate from the strong form of the governing equations and instead incorporate the weak form into their loss function. Traditionally, this approach is critical where high-order differentiability of the system is not guaranteed. In the context of PINNs, this approach reduces the load of the auto differentiation calculations and thus accelerates the PINNs computations. Procedurally, VPINNs use test functions and quadrature points instead of a random collocation of points to compute the integrals and consequently, the variational residuals. This replacement of the training dataset by quadrature points has demonstrated a reduction in the training time. Although VPINNs require a mesh to define the integrals and basis functions, Mesh-Free Variational PINNs (MF-VPINNs) have been developed since to address this issue [44].

Another variant of PINNs that utilizes the integral form of the conservation laws is called the Integral PINNs (IP-INNs) [45]. This variant has immense utility for problems that include shock discontinuities where the differential form of the governing equation does not hold. Finite Basis PINNs (FBPINNs) [46] methods combine PINNs with concepts from finite basis functions (e.g., Fourier series, wavelets) to improve the representation capacity and efficiency, particularly for oscillatory or multi-scale problems.

Acceleration of PINNs training can also be achieved through Adaptive PINNs (APINNs) [47], which dynamically adapt various aspects of the PINNs during training. This can involve Adaptive Sampling, which adjusts the distribution of collocation points based on the PDE residual or solution error to focus computational resources on challenging regions. Adaptive Weighting strategy dynamically adjusts the weights of different terms in the loss function (e.g., physics loss, data loss, boundary condition loss) to balance their contributions and improve convergence. Adaptive Activation Functions can change their parameters during training to better fit the solution.

Another line of research close to PINNs is that of Operator Learning methods [48]. While distinct from standard PINNs, these are closely related and represent a powerful class of "operator learning" methods. Instead of learning a specific solution to a PDE, they learn the operator that maps an input function (e.g., initial condition, boundary condition, source term) to the solution function. Once trained, a PINO/DeepONet can rapidly predict solutions for any new input function without retraining, making them incredibly efficient for many-query problems. In particular, DeepONets introduced in [49] are inspired by the Universal Approximation Theorem of Neural Networks and attempt to learn the solution operator. A popular physics-informed variation of this operator called Physics-Informed Neural Operator (PINO) was

introduced in [50]. PINO makes use of Fourier Neural Operator (FNO) [51] and combines training data and physics constraints to learn the solution operator for a given family of parametric PDEs.

Physics-Informed Neural Networks (PINNs) present unique challenges when applied to supersonic flows due to the difficulty of computing gradients across shock discontinuities. Nevertheless, they have demonstrated the capability to capture shock structures and discontinuities without relying on traditional numerical solvers [52]. PINNs have also proven effective for modeling multi-phase flows involving shocks, without the need for explicit shock-capturing techniques [53]. To improve robustness in shock-dominated regimes, a hybrid approach combining mesh-aware training and shock-aware viscosity control within the PINNs framework was proposed in [54]. Furthermore, PINNs have been shown to match the performance of traditional shock-fitting and shock-capturing solvers in predicting both continuous and discontinuous flow features in nozzle geometries, enabling data-free, high-fidelity CFD using neural approaches [55, 56]. Recent work also introduced a reinitialization strategy tailored for stiff problems, such as high-Reynolds-number flows, enhancing the convergence and accuracy of PINNs training in such regimes [57].

4.3 PINNs to solve PDEs

In this section, we shall see the procedure to solve PDEs using PINNs. Let's consider the 1D heat equation:

$$\frac{\partial u}{\partial t} = \alpha \frac{\partial^2 u}{\partial x^2}, \quad x \in [0,1],\, t \in [0,T]$$

With the initial condition:

$$u(x,0) = u_0(x)$$

and boundary conditions:

$$u(0,t) = u_L(t), \quad u(1,t) = u_R(t)$$

As per the universal approximation theorem, neural networks can approximate any continuous function. So, we can approximate the solution of the heat equation using a neural network:

$$u(x,t) \approx u_\theta(x,t)$$

where θ denotes the parameters (weights and biases) of the neural network. The random weight and bias cannot satisfy the PDEs, so we will get some residue, which can be written as

$$\mathcal{R}(x,t) = \frac{\partial u_\theta}{\partial t} - \alpha \frac{\partial^2 u_\theta}{\partial x^2}$$

The physics-based loss (PDE loss) over all sample points can be written as

$$\mathcal{L}_{\text{PDE}} = \frac{1}{N_f} \sum_{i=1}^{N_f} \left| \mathcal{R}(x_f^i, t_f^i) \right|^2$$

where $\{(x_f^i, t_f^i)\}_{i=1}^{N_f}$ are collocation points in the interior domain.

$$\mathcal{L}_{\text{IC}} = \frac{1}{N_0} \sum_{i=1}^{N_0} \left| u_\theta(x_0^i, 0) - u_0(x_0^i) \right|^2$$

The PDE can give a unique solution only when we apply the initial and boundary conditions.

$$\mathcal{L}_{\text{BC}} = \frac{1}{N_b} \sum_{i=1}^{N_b} \left(\left| u_\theta(0, t_b^i) - u_L(t_b^i) \right|^2 + \left| u_\theta(1, t_b^i) - u_R(t_b^i) \right|^2 \right)$$

The total loss is a weighted sum of all loss components:

$$\mathcal{L}_{\text{total}} = \lambda_{\text{PDE}} \mathcal{L}_{\text{PDE}} + \lambda_{\text{IC}} \mathcal{L}_{\text{IC}} + \lambda_{\text{BC}} \mathcal{L}_{\text{BC}}$$

where $\lambda_{\text{PDE}}, \lambda_{\text{IC}}, \lambda_{\text{BC}}$ are weights that can be tuned to balance the contributions. The network parameters θ are optimized using gradient-based methods:

$$\theta^* = \arg\min_\theta \mathcal{L}_{\text{total}}(\theta)$$

4.4 Adaptive weight and viscosity PINNs architecture (PINNs-AWV)

Adaptive weight and viscosity PINNs architecture is a sophisticated neural network architecture designed to handle shocks and discontinuity was introduced in [28]. Most of the problems involving shocks and discontinuities are modeled using differential equations. When we calculate the derivative across the shocks, the residue of the PDEs has a high value, which will destabilize the solver. Some of the ways to handle this are by reducing the weights at these regions or adding artificial viscosity to the solver to spread the shocks over several grid points to reduce the magnitude of the derivatives across the shocks. This architecture uses both features to stabilize the solver. We have used 1-D Burgers' Equation to explain the mathematical details of the the PINNs-AWV architecture. The 1D Burgers' Equation is

$$\frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} = 0 \tag{56}$$

To smooth the shock produced by the Burgers equation, we add an artificial viscosity term to the Burgers equation to stabilize the solver, but we don't know the optimal viscosity to stabilize the solver. If we add a large amount, this will reduce the shock resolution property of the scheme. If we add too much or not enough will destabilize the solver [58]. In traditional numerical methods, we test the solvers on several test cases and find the optimal viscosity that works. We can also learn the optimal viscosity, which stabilize the solver using the neural network [25], [59]. The Burgers equation with artificial viscosity can be written as:

$$\frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} = \nu\frac{\partial^2 u}{\partial x^2}$$

where $x \in [-1, 1]$, $t \in [0, T]$, and ν is the artificial viscosity co-efficent. We approximate the solution as:

$$u(x, t) \approx u_\theta(x, t)$$

with neural network parameters θ. Where ν is the trainable parameter modeled via a sub-network. Because the Burgers equation does not have viscosity, we should minimize the viscosity. The PINNs will find the optimal viscosity to stabilize the solver without training data. So the viscous loss can be written as

$$\mathcal{L}_\nu = \nu^2$$

The PDE residual becomes:

$$\mathcal{R}(x, t) = \frac{\partial u_\theta}{\partial t} + u_\theta\frac{\partial u_\theta}{\partial x} - \nu(x, t)\frac{\partial^2 u_\theta}{\partial x^2}$$

Considering initial and boundary losses

$$\mathcal{L}_{\text{IC}} = \frac{1}{N_0}\sum_{i=1}^{N_0}\left|u_\theta(x_0^i, 0) - u_0(x_0^i)\right|^2$$

$$\mathcal{L}_{\text{BC}} = \frac{1}{N_b}\sum_{i=1}^{N_b}\left(|u_\theta(-1, t_b^i) - u_L(t_b^i)|^2 + |u_\theta(1, t_b^i) - u_R(t_b^i)|^2\right)$$

When we use weights λ_{PDE} that adapt based on gradient norms:

$$\lambda_{PDE} = \frac{1}{\|\nabla_\theta \mathcal{L}_i\| + \epsilon}$$

The total loss is

$$\mathcal{L}_{\text{total}} = \tilde{\lambda}_{\text{PDE}}\mathcal{L}_{\text{PDE}} + \tilde{\lambda}_{\text{IC}}\mathcal{L}_{\text{IC}} + \tilde{\lambda}_{\text{BC}}\mathcal{L}_{\text{BC}} + \tilde{\lambda}_\nu\mathcal{L}_\nu$$

All parameters, including neural network weights θ and possibly $\nu(x, t)$, are optimized by minimizing the total loss:

$$\theta^*, \nu^* = \arg\min_{\theta,\nu} \mathcal{L}_{\text{total}}(\theta, \nu)$$

- Adaptive weights help to reduce the magnitude of PDE losses across the shocks and stabilizes the solver.
- Adaptive viscosity will try to find the optimal viscosity which stabilizes the solver.

Figure 3 shows the steps used in the PINNs-AWV architecture.

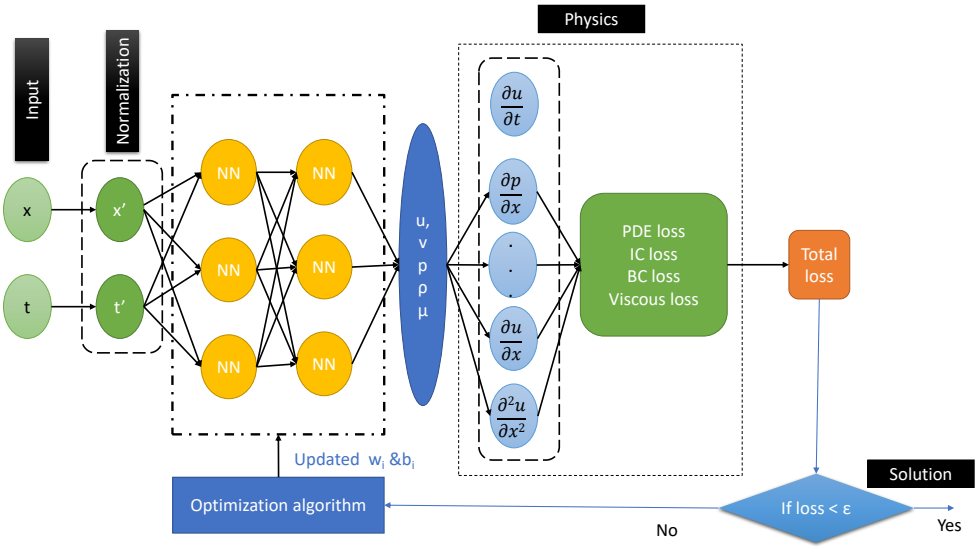


Figure 3: Steps used in Adaptive weight and viscosity PINNs architecture

5 Results

In the previous sections we found numerical methods need are very sensitive to governing equation and discretization. In this section, we shall study whether PINNs also exhibit similar issue by testing it on few problems where numerical methods shows different results for conservative and non-conservative forms. To make a fair comparison we have included the non-conservative equation with artificial viscosity for problem having shocks.

5.1 Shallow Water Equations with Variable Bottom Surface

The one-dimensional shallow water equations with variable bathymetry that cannot be written in "purely" conservative form. We already discussed in subsection 3.5, where traditional numerical schemes need well-balanced property to accurately solve this problem. We have used the same discretization we used in subsection 3.5 for this problem. A reference solution is computed using a conservative finite-volume method implemented in PyClaw [60]. The equations are solved in conservative variables (h, hu) using a Roe-type approximate Riemann solver combined with an operator-split treatment of the bathymetry source term [61]. Reflective boundary conditions are imposed at both ends of the domain.

The finite-volume method preserves total mass to machine precision,

$$M(t) = \int_0^L h(x,t)\,dx = \text{const}, \tag{57}$$

while exhibiting a small, physically consistent decay of total mechanical energy due to numerical dissipation and limiter effects. The resulting solution serves as a reference benchmark for assessing the qualitative behavior and conservation properties of the physics-informed neural network formulation. The physics-informed neural network (PINNs) directly approximates the solution fields $h(x,t)$ and $u(x,t)$. The governing equations are enforced in conservative form by minimizing the residuals of the balance laws at randomly sampled spatio-temporal collocation points. In addition to the standard PDE, initial-condition, and boundary-condition losses, a global mass constraint is introduced to penalize deviations from total mass conservation,

$$\mathcal{L}_{\text{mass}} = \left(\int_0^L h(x,t)\,dx - \int_0^L h(x,0)\,dx \right)^2. \tag{58}$$

This term weakly enforces global conservation without introducing explicit numerical fluxes or discretization stencils. Since the solution remains smooth, errors are quantified using an L_2 norm. To remove contamination by the mean water level, we compare the free-surface perturbation,

$$\eta'(x,t) = h(x,t) + b(x) - H_0. \tag{59}$$

The relative L_2 error at final time T is defined as

$$\|\eta'_{\text{PINN}} - \eta'_{\text{ref}}\|_{L^2} = \left(\frac{1}{L} \int_0^L \left(\eta'_{\text{PINN}}(x,T) - \eta'_{\text{ref}}(x,T) \right)^2 dx \right)^{1/2}. \tag{60}$$

For shock-dominated problems considered in subsequent sections, L_1 norms are employed instead to avoid excessive sensitivity to localized discontinuities.

Here, we have used the same discretization we used for subsection 3.5.1. Figure 4 compares the free-surface elevation and mass conservation behavior obtained using a physics-informed neural network (PINNs), a conservative finite-volume method (FVM), and a non-conservative finite-difference method (FDM) for the shallow water equations with variable bathymetry. Figure 4a shows the free-surface elevation at the final time t = 5.0 s. The conservative finite-volume solution closely follows the expected physical behavior, retaining small-amplitude oscillations associated with wave propagation and reflection over the variable bottom. The PINNs solution reproduces the large-scale structure of the free surface while exhibiting a smoother profile, reflecting the regularizing effect of optimization-based training. In contrast, the non-conservative finite-difference solution displays a significant distortion of the free surface, with an artificial amplification of the water level that is inconsistent with the underlying physics.

The PINNs solution reproduces the large-scale structure of the free surface while exhibiting a smoother profile, reflecting the regularizing effect of optimization-based training. **Notably, this result is obtained without the introduction of numerical fluxes, Riemann solvers, limiters, or specialized well-balanced discretizations. Instead, the PINNs relies solely on the governing balance laws and a global mass conservation constraint**, making the implementation conceptually simple compared to classical conservative solvers.

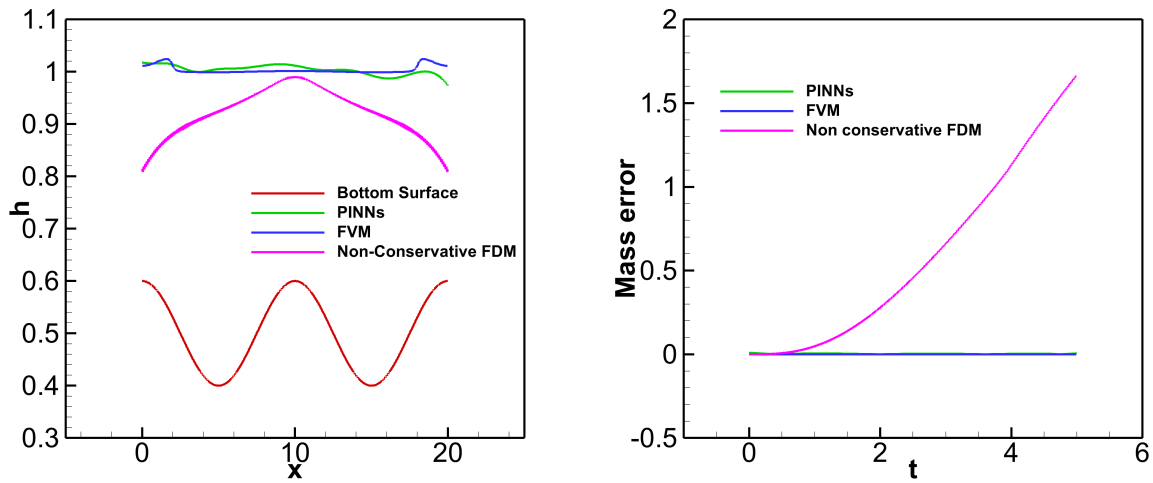
In contrast, the non-conservative finite-difference solution displays a significant distortion of the free surface, with an artificial amplification of the water level that is inconsistent with the underlying physics. This behavior is directly linked to the lack of a conservative formulation and illustrates the cumulative effect of local discretization errors on global invariants. Figure 4b reports the evolution of the global mass error. The finite-volume method preserves mass to machine precision throughout the simulation, as expected from its conservative formulation. The PINNs solution also maintains bounded mass error without long-time drift, demonstrating that global conservation can be effectively enforced through integral constraints, despite the absence of explicit numerical fluxes. In sharp contrast, the non-conservative finite-difference scheme exhibits a monotonic growth in mass error, leading to a substantial accumulation of artificial mass over time. Table 4 shows the L_2 error of different scheme for this simulation. From this it is clear that PINNs solution is independent on the governing equation form for this problem.

These results highlight an important methodological distinction. While finite-volume methods achieve conservation through carefully designed numerical structures and problem-specific treatments, physics-informed neural networks can enforce the same global invariant through a comparatively simple optimization framework. At the same time, the smoother PINNs solution indicates that conservation alone is not sufficient to recover detailed hyperbolic wave dynamics, underscoring the complementary strengths and limitations of optimization-based and discretization-based approaches.

This experiment demonstrates that conservation of global invariants can be imposed in physics-informed neural networks without explicit flux discretization. However, conservation alone is not sufficient to guarantee accurate phase-resolved wave dynamics, underscoring a fundamental difference between numerical discretization and optimization-based solution strategies. These findings motivate the subsequent investigation of shock-dominated problems, where the distinction between conservative and non-conservative formulations becomes even more pronounced.

Table 4: Comparison of L_2 error in free surface elevation for different methods for SWE with no initial disturbance.

Method	L_2 **Error (Free Surface Elevation)**
Numerical Methods	
Non-Conservative Scheme	9.2899×10^{-2}
Conservative Scheme	1.0097×10^{-1}
Conservative Scheme (WB)	8.7101×10^{-15}
Physics-Informed Neural Networks	
PINNs (Non-Conservative)	4.7702×10^{-4}
PINNs (Conservative)	6.9405×10^{-3}



(a) Free-surface elevation

(b) Mass error

Figure 4: Solution of shallow water equation at t = 5.0 s with small initial disturbance

5.2 Burgers equation on discontinuous initial condition

In this test case, we have used a discontinuous initial condition, which is

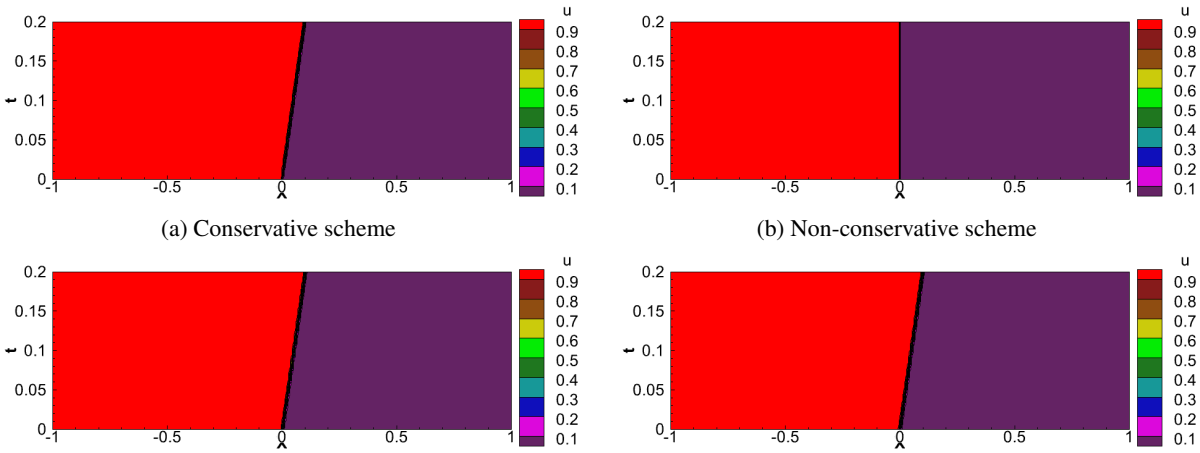
$$u(x,0) = \begin{cases} 1.0, & \text{if } x \leq 0 \\ 0.0, & \text{if } x > 0 \end{cases} \tag{61}$$

The same discretization procedure used in the subsection 3.4 is used here but we have used only 101 grid points for the simulation. The figure 5a shows the solution obtained using upwind scheme based on conservative discretization. Figure 5b shows the solution obtained using upwind scheme based on non-conservative discretization. From this, we can observe that the solution is *not propagating* when we use non-conservative discretization on discontinuous initial condition. Figure 5c and 5d shows the solution obtained using PINNs using the strong form of the conservative and non-conservative equations. From this, we can observe that PINNs produced the same result regardless of the form of the governing equation we used. Figure 6a shows the solution of this test case at t=0.2. From this, we can observe that numerical methods based on a non-conservative scheme are unable to predict the shock location.

We already studied that researchers used the artificial viscosity procedure to obtain the solution. So we added a scalar dissipation value of 0.001 and solved the problem. Figure 6b shows the solution of the Burgers equation solved with a scalar viscosity value of 0.001 using the non-conservative form of the Burgers equation. The solution at t=0.12 is shown in figure 6b. The non-conservative scheme is unable to predict shock speed when we don't use artificial viscosity, but is able to predict the shock speed when we add artificial viscosity. The magnitude of the viscosity and time step used is depends on the number of grid points and the initial condition we used. Though the non-conservative scheme with viscosity is able to predict the shock location, the scheme is very diffusive. When we reduce the magnitude of the viscosity, the scheme may become unstable. But in PINNs, it automatically find the optimal viscosity required to stabilize the solver. L_1 error and mass defect calculated for different scheme with respect to the exact solution is tabulated in table 5.The results indicate that non-conservative formulations whether implemented via PINNs or traditional numerical discretizations—produce substantially larger errors than conservative schemes. While the non-conservative PINNs shows higher error relative to conservative approaches, it nevertheless captures the shock location effectively and achieves lower error than the viscously stabilized non-conservative finite-difference method.

Table 5: Final-time diagnostics for Burgers' equation at $t = 0.2$.

Scheme	L_1 Error	Mass Defect
Numerical Methods		
Conservative	1.7226×10^{-3}	-1.5401×10^{-14}
Non-Conservative	1.0101×10^{-1}	-1.0101×10^{-1}
Non-Conservative-μ	4.6573×10^{-2}	-6.8782×10^{-3}
Physics-Informed Neural Networks		
Conservative PINNs	4.3307×10^{-3}	-1.8086×10^{-5}
Non-Conservative PINNs	4.8519×10^{-3}	-1.8893×10^{-3}



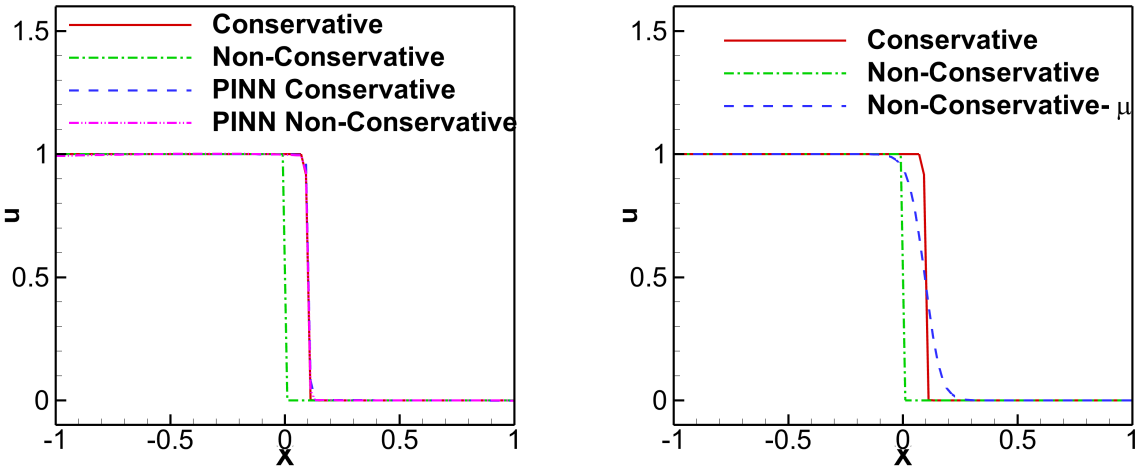
(a) Conservative scheme

(b) Non-conservative scheme

(c) PINNs-conservative

(d) PINNs-non-conservative

Figure 5: Solution of Burgers equation with discontinuous initial condition



(a) Numerical and PINNs

(b) Numerical schemes

Figure 6: Solution of Burgers equation at t = 0.2 s

5.3 Sod Shock Tube Problem

To assess the robustness of PINNs-AWV, we consider the Euler equations, a set of coupled non-linear hyperbolic equations. In this study, we examine the system in both conservative and non-conservative forms. The efficacy of these formulations has been demonstrated in previous literature for the Sod shock tube [62, 1] and supersonic wedge flow problems [63]. The Euler equations in conservative form are given by:

$$\frac{\partial}{\partial t}\begin{bmatrix}\rho\\ \rho u\\ E\end{bmatrix}+\frac{\partial}{\partial x}\begin{bmatrix}\rho u\\ \rho u^2+p\\ u(E+p)\end{bmatrix}=0 \tag{62}$$

where ρ, u, p, and E denote density, velocity, pressure, and total energy per unit volume ($E=\frac{p}{\gamma-1}+\frac{1}{2}\rho u^2$, $\gamma=1.4$). The initial conditions for the Sod shock tube problem are:

$$(\rho,u,p)(x,0)=\begin{cases}(1.0,\ 0.0,\ 1.0), & \text{if } x<0.5\\ (0.125,\ 0.0,\ 0.1), & \text{if } x>0.5\end{cases} \tag{63}$$

The non-conservative form is expressed as:

$$\begin{aligned}\frac{\partial\rho}{\partial t}+u\frac{\partial\rho}{\partial x}+\rho\frac{\partial u}{\partial x}&=0\\ \frac{\partial u}{\partial t}+u\frac{\partial u}{\partial x}+\frac{1}{\rho}\frac{\partial p}{\partial x}&=0\\ \frac{\partial p}{\partial t}+u\frac{\partial p}{\partial x}+\gamma p\frac{\partial u}{\partial x}&=0\end{aligned}$$

To handle this form numerically, we implement both a standard non-conservative- μ scheme and a path-conservative scheme based on Dal Maso-LeFloch-Murat (DLM) theory [18]. The problem is solved using 100 uniform spatial and 100 temporal collocation points for PINNs-AWV [28]. Numerical solutions are obtained on a 100-point grid using an HLLC solver, minmod limiter, and HRK time integration [64]. The pressure contours are presented in Figure 8. While numerical methods resolve the shock (Figures 7b and 7d), they exhibit significant diffusion. In contrast, the PINNs-AWV solutions (Figures 7c–7f) produce sharper profiles. Table 6 summarizes the L_1 errors and mass defect at $T=0.12$ s. Notably, the numerical path-integral method shows improved mass conservation over the standard non-conservative scheme, yet PINNs maintain a lower error profile across the board at early flow times.

5.3.1 Effect of PDE Formulation on Shock Speed in Adaptive-Viscosity PINNs

Despite the success at early flow times, extended simulations reveal a fundamental limitation of non-conservative PINNs. Figure 9b illustrates the pressure solution at $T=0.5$ s. While PINNs applied to conservative PDEs capture the exact shock location, the non-conservative formulation fails to capture the correct shock speed. Interestingly, this failure is unique to system of equations; for the scalar Burgers equation ($u_t+uu_x=0$), the non-conservative PINN accurately captures shock propagation even at $T=4$ s (Figure 9a). This is because the analytical identity $uu_x=(0.5u^2)_x$ holds for smooth neural approximations, making the formulations equivalent during training. However, for the Euler system, regularizing the primitive-variable formulation (V) with adaptive viscosity ν introduces a non-conservative term when transformed back to conservative variables (U):

$$U_t+F(U)_x=(\nu U_x)_x-\nu\frac{\partial^2 U}{\partial V^2}(V_x)^2 \tag{64}$$

This additional term modifies the effective viscous structure within the shock layer. As $\nu\to 0$, the solutions of the two systems may converge to different weak limits. This confirms that preserving conservative formulations is critical when applying PINNs to hyperbolic systems with discontinuities, as different regularizations of non-conservative systems converge to fundamentally different weak solutions [18].

5.3.2 Path-Integral Formulation for Non-Conservative PINNs

A fundamental challenge in solving the Euler equations in the non-conservative (primitive) form $\mathbf{V}=[\rho,u,p]^T$ is that the product $\mathbf{A}(\mathbf{V})\mathbf{V}_x$ is not well-defined in the sense of distributions when shocks are present. To address this, we employ the Dal Maso-LeFloch-Murat (DLM) theory [18], which defines non-conservative products across discontinuities by integrating along a family of paths $\psi(\tau;\mathbf{V}_L,\mathbf{V}_R)$ for $\tau\in[0,1]$.

In this study, we utilize a linear path in the phase space:

$$\psi(\tau) = \mathbf{V}_L + \tau(\mathbf{V}_R - \mathbf{V}_L) \tag{65}$$

The total contribution of the non-conservative product across a localized gradient (the jump) is given by the path integral:

$$\mathcal{P}(\mathbf{V}_L, \mathbf{V}_R) = \int_0^1 \mathbf{A}(\psi(\tau)) \frac{\partial \psi(\tau)}{\partial \tau} d\tau \tag{66}$$

where $\mathbf{A}(\mathbf{V})$ is the system Jacobian in primitive variables.

5.3.3 Recovery of Correct Shock Speed via Path-Integral Formulation

Figure 9b illustrates this discrepancy at $T = 0.5$ s, where the non-conservative formulation with standard artificial viscosity diverges significantly from the exact solution. However, by employing the Path-Integral (PI-PINN) approach, we successfully recover the correct shock location and propagation speed, matching the accuracy of the conservative formulation. This success is rooted in the treatment of the non-conservative product $\mathbf{A}(\mathbf{V})\mathbf{V}_x$. In classical theory [18], the weak solution of a non-conservative system depends on the viscous regularization path. A standard neural network, when regularized by adaptive viscosity ν, tends to satisfy:

$$\mathbf{V}_t + \mathbf{A}(\mathbf{V})\mathbf{V}_x = (\nu \mathbf{V}_x)_x. \tag{67}$$

As shown in the previous analysis, transforming this back to conservative variables $\mathbf{U}$ introduces a source term $\nu \frac{\partial^2 \mathbf{U}}{\partial \mathbf{V}^2}(\mathbf{V}_x)^2$ that does not vanish in the shock layer as $\nu \to 0$. This term shifts the Rankine-Hugoniot jump conditions, leading to the observed shock speed error. The Path-Integral approach resolves this by explicitly enforcing the jump condition through the Dal Maso-LeFloch-Murat (DLM) framework. By integrating the Jacobian $\mathbf{A}(\mathbf{V})$ along a linear path in state space, the PI-PINN loss:

$$Loss_{path} = \left\| \left(\int_0^1 \mathbf{A}(\psi(\tau)) d\tau \right) \Delta \mathbf{V} - \Delta \mathbf{F} \right\|^2 \tag{68}$$

effectively constrains the neural network to satisfy the conservation properties of the system, even when expressed in primitive variables. Our results demonstrate that for the Sod shock tube problem at $T = 0.5$ s, the PI-PINN and the path-conservative numerical method both yield solutions that are consistent with the exact Riemann solution, effectively bridging the gap between non-conservative formulations and physical conservation laws. The numerical approximation of non-conservative hyperbolic systems has been rigorously studied using the path-conservative framework introduced by Parés [65], which generalizes conservative finite volume schemes and Riemann solvers to systems containing non-conservative products. However, subsequent studies have shown that even when the theoretically correct path is used, path-conservative schemes may still produce incorrect shock solutions due to discretization errors and numerical viscosity effects [66].

5.3.4 Implementation in PINNs-AWV

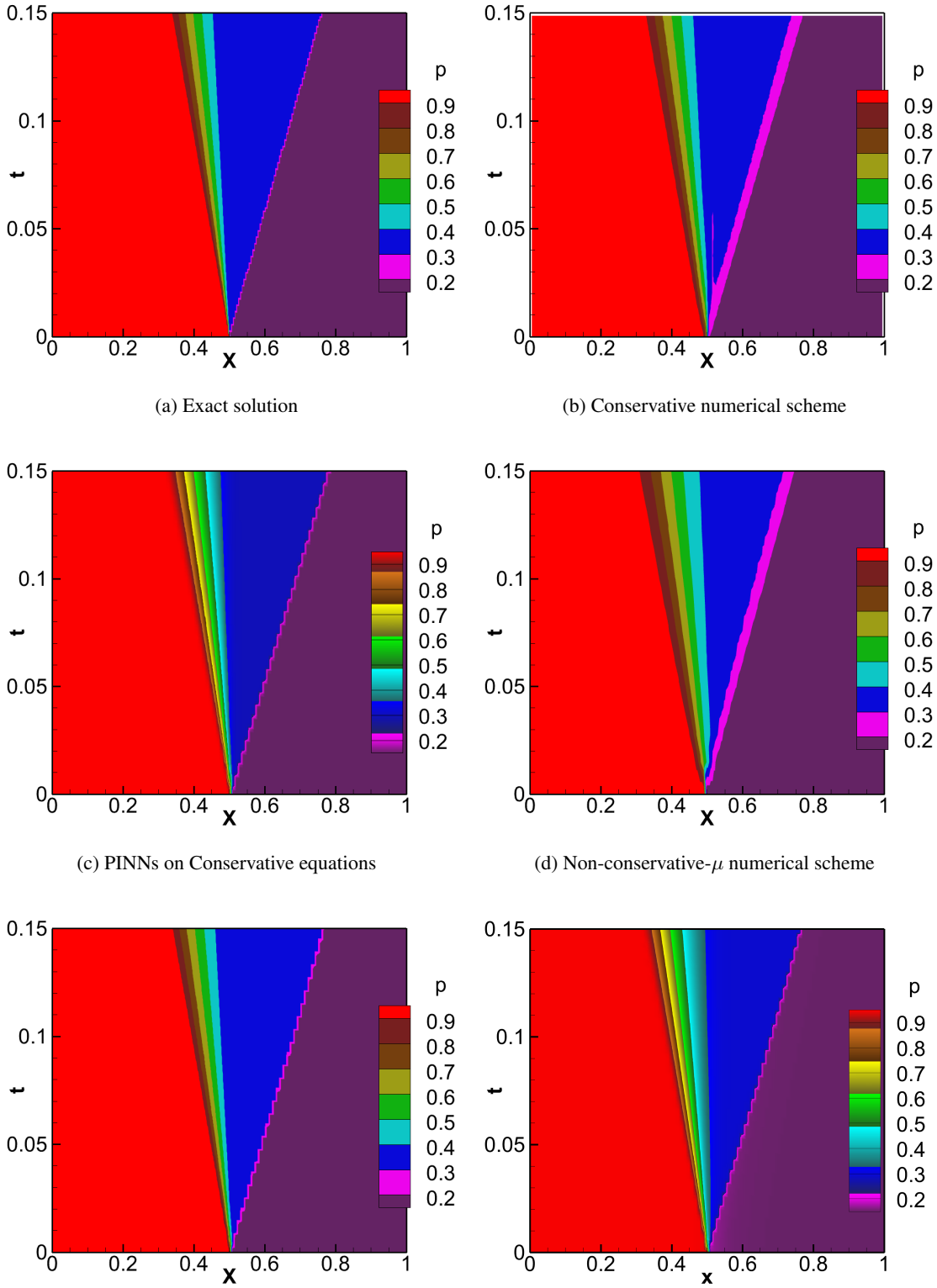
To implement this within the PINN framework, we modify the residual loss at each collocation point (x, t). We define a localized stencil by perturbing the spatial coordinate by a small parameter ϵ, yielding left and right states $\mathbf{V}_L = \mathbf{V}(x - \epsilon, t)$ and $\mathbf{V}_R = \mathbf{V}(x + \epsilon, t)$. The path integral is then approximated using 3-point Gauss-Legendre quadrature:

$$\mathcal{I}_{path} \approx \sum_{i=1}^{3} \omega_i \mathbf{A}(\psi(\tau_i))(\mathbf{V}_R - \mathbf{V}_L) \tag{69}$$

where τ_i and ω_i are the quadrature nodes and weights, respectively. The Path-Integral loss ($Loss_{path}$) is defined as the squared difference between this integral and the conservative flux jump:

$$Loss_{path} = \|\mathcal{I}_{path} - [\mathbf{F}(\mathbf{V}_R) - \mathbf{F}(\mathbf{V}_L)]\|^2 \tag{70}$$

This formulation ensures that the neural network approximation satisfies the generalized Rankine-Hugoniot conditions required for consistent shock capturing in non-conservative systems. The pressure contours for the Sod shock tube problem are presented in Figure 8. We compare the results of PINNs-AWV (Conservative, Non-conservative, and Path-Integral) against traditional numerical schemes (Conservative FV, Non-conservative-μ, and Path-conservative). At $T = 0.12$ s, Table 6 shows that the Path-Integral PINN significantly improves the resolution of the shock compared to the standard non-conservative numerical method. Specifically, the numerical path-integral method achieves a mass defect of 2.48×10^{-4}, which is an order of magnitude improvement over the standard non-conservative-μ scheme



(a) Exact solution

(b) Conservative numerical scheme

(c) PINNs on Conservative equations

(d) Non-conservative-μ numerical scheme

(e) PINNs on non-conservative equations

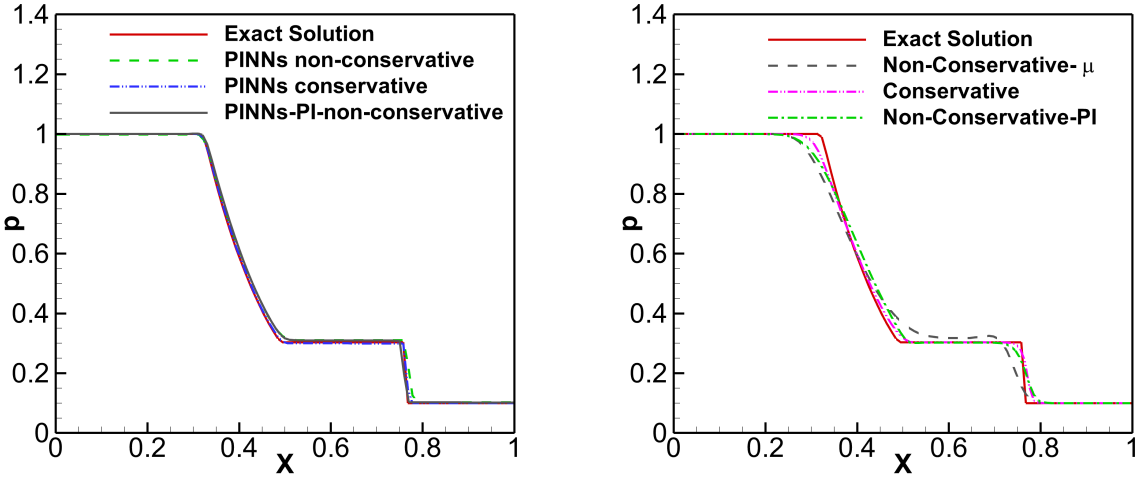
(f) PINNs on non-conservative equations with path integral

Figure 7: Pressure solution of Sod shock tube problem over time

Table 6: Updated primitive-variable errors and mass defect for the Sod shock tube problem at T = 0.12 s.

Scheme	$L_1(\rho)$	$L_1(u)$	$L_1(p)$	Mass Defect
		Numerical Methods		
Conservative FV	8.3393×10^{-3}	1.7016×10^{-2}	7.6215×10^{-3}	-6.0587×10^{-13}
Non-conservative-μ	1.8629×10^{-2}	3.9520×10^{-2}	2.0391×10^{-2}	-2.8099×10^{-3}
Non-conservative-PI	1.4340×10^{-2}	2.8636×10^{-2}	1.3445×10^{-2}	2.4829×10^{-4}
		Physics-Informed Neural Networks		
PINNs Conservative	6.6526×10^{-3}	6.5373×10^{-3}	1.6889×10^{-2}	3.3238×10^{-3}
PINNs Non-conservative	7.0354×10^{-3}	6.4271×10^{-3}	1.0955×10^{-2}	4.0261×10^{-3}
PINNs-PI Non-conservative	1.7529×10^{-2}	3.7016×10^{-2}	1.9785×10^{-2}	-2.8099×10^{-3}

(-2.81×10^{-3}). While all PINN formulations show negligible differences in error at this early stage, the path-integral formulation provides a mathematically consistent framework for handling the primitive variables. However, as flow time increases to $T = 0.5$ s (Figure 9b), the non-conservative formulations using artifical viscosity didn't produce correct shock speed but with path integral loss function, PINNs able to give correct shock speed.



(a) Using PINNs

(b) Using numerical methods

Figure 8: Pressure solution of Sod shock tube problem at T =0.15

5.4 Supersonic flow over wedge

The governing equations used are the steady-state two-dimensional Euler equations, which can be written in conservative differential form as:

$$\frac{\partial F}{\partial x} + \frac{\partial G}{\partial y} = 0 \tag{71}$$

where the flux vectors are:

$$F = \begin{pmatrix} \rho u \\ \rho u^2 + p \\ \rho u v \\ u(E+p) \end{pmatrix}, \quad G = \begin{pmatrix} \rho v \\ \rho u v \\ \rho v^2 + p \\ v(E+p) \end{pmatrix}. \tag{72}$$

The total energy per unit volume is $E = \rho e + \frac{1}{2}\rho(u^2 + v^2)$. The equation of state for an ideal gas is $p = (\gamma - 1)\rho e$, which, expressed in terms of total energy E, is:

$$p = (\gamma - 1)\left(E - \frac{1}{2}\rho(u^2 + v^2)\right). \tag{73}$$

Algorithm 1 PINNs-AWV for Sod shock tube problem

Initialize PINNs Model
Define Initial Condition Function $IC(x)$
Generate spatio-temporal training data $x_{int_train} = (t, x)$ for PDE residuals
Generate initial condition data $x_{ic_train} = (0, x)$ and corresponding $\rho_{ic_train}, u_{ic_train}, p_{ic_train}$ using $IC(x)$
function LossPDE(x_{int_train})
    Predict $\rho, p, u, \nu_i = NeuralNetwork(x_{int_train})$
    Compute first derivatives: $\rho_t, \rho_x, u_t, u_x, p_t, p_x$ using
    Compute second derivatives: $\rho_{xx}, u_{xx}, p_{xx}$
    Calculate adaptive weight term $d = 0.12 \cdot (|u_x| - u_x) + 1$
    Calculate viscosity $\nu = \nu_i^2$
    Formulate PDE residuals:
    $L_C = (\rho_t + u\rho_x + \rho u_x - \nu\rho_{xx})/d$
    $L_M = (u_t + uu_x + (1/\rho)p_x - \nu u_{xx})/d$
    $L_E = (p_t + up_x + \gamma p u_x - \nu p_{xx})/d$
    return mean(L_C^2) + mean(L_M^2) + mean(L_E^2) + mean(ν^2)
end function
function LossIC($x_{ic_train}, \rho_{ic_train}, u_{ic_train}, p_{ic_train}$)
    Predict $\rho_{ic_nn}, p_{ic_nn}, u_{ic_nn} = NeuralNetwork(x_{ic_train})$
    return mean$((u_{ic_nn} - u_{ic_train})^2)$ + mean$((\rho_{ic_nn} - \rho_{ic_train})^2)$ + mean$((p_{ic_nn} - p_{ic_train})^2)$
end function
$L_{\text{pde}} = \text{LossPDE}(M, x_{int_train})$
$L_{\text{ic}} = \text{LossIC}(M, x_{ic_train}, \rho_{ic_train}, u_{ic_train}, p_{ic_train})$
$L_{\text{total}} = L_{\text{pde}} + 1 \cdot L_{\text{ic}}$
Compute gradients of L_{total} with respect to model parameters
return L_{total}
Phase 1: Adam Optimization
Phase 2: L-BFGS Optimization
Trained model: $NeuralNetwork(t, x)$=u,ρ,p

Alternatively, the non-conservative (primitive variable) form of the steady-state Euler equations can be expressed in terms of the primitive variable vector $W = [\rho, u, v, p]^T$ as:

$$A\frac{\partial W}{\partial x} + B\frac{\partial W}{\partial y} = 0 \tag{74}$$

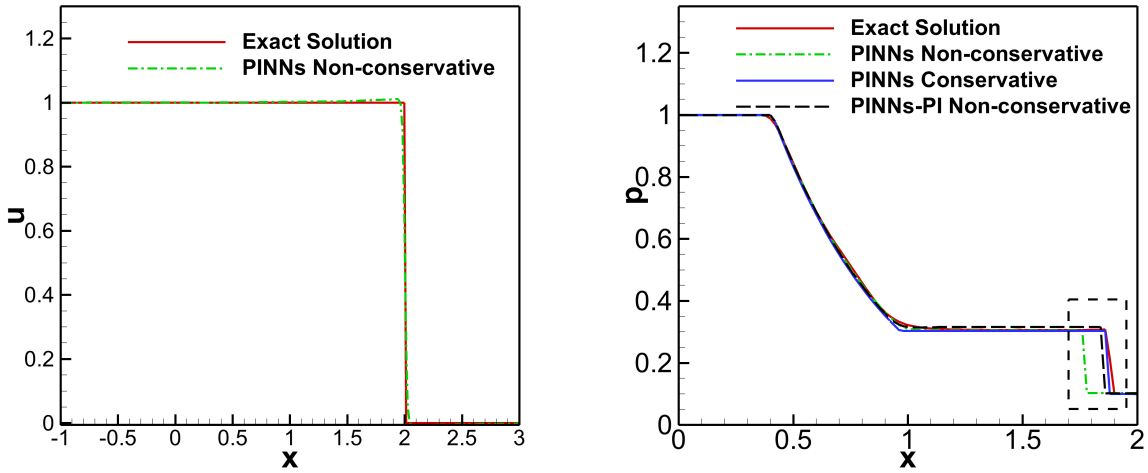
where the coefficient matrices A and B are given by:

$$A = \begin{pmatrix} u & \rho & 0 & 0 \\ 0 & u & 0 & \frac{1}{\rho} \\ 0 & 0 & u & 0 \\ 0 & \gamma p & 0 & u \end{pmatrix}, \quad B = \begin{pmatrix} v & 0 & \rho & 0 \\ 0 & v & 0 & 0 \\ 0 & 0 & v & \frac{1}{\rho} \\ 0 & 0 & \gamma p & v \end{pmatrix}. \tag{75}$$

In this test case, supersonic flow with an inflow Mach number of 2 is simulated over a wedge with a 10° angle. For the conservative formulation, we used Euler time integration with first-order reconstruction scheme with Lax-Fridrich scheme as Riemann solver. The goal of the present work is to study whether the non-conservative scheme can handle shocks not the accuracy. For numerical methods we rotated the flow 10 degree so that we can use the advantage of Cartesian grid for this problem without special treatment for wall boundaries. We observed some mass defect in conservative scheme because we evaluate mass conservation by adding all inflow and outflow mass over the boundaries. Because we use transient simulation and didn't use the characteristic boundary condition we got non-zero mass defect for conservative scheme.

We employed 10,000 collocation points in the PINNs simulations, utilizing an adaptive-weight and viscosity-based PINNs architecture tailored for this problem. The exact solution is presented in Figure 10a. A slip boundary condition was applied along the wedge walls, with specified inflow parameters imposed at the inlet.

Figures 10c and 10d display the pressure contours obtained from PINNs simulations using the conservative and non-conservative forms of the governing equations, respectively. For comparison, Figure 10b shows the pressure contours computed with a conventional conservative numerical method employing the MUSCL scheme. More details about the numerical discretization can be found in [63].



(a) Burgers solution at T = 4 s

(b) Pressure solution of Sod shock tube solution at T = 0.5 s

Figure 9: Solution at long flow time

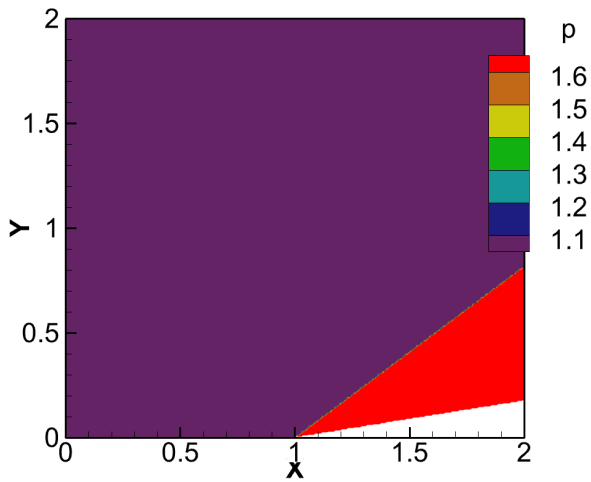
In our PINNs simulations, shocks are resolved with approximately three grid points, consistent with the findings of Neelan et al. [28]. Notably, all methods successfully capture the shock structures. Furthermore, as demonstrated by the inclusion of the non-conservative Euler equations for PINNs in this comparison, the PINNs framework accurately captures the correct shock location for this problem without the positional discrepancies typically encountered in standard non-conservative schemes. Figure 11 compares the line plots of pressure solutions obtained from PINNs-AWV simulations using both the conservative and non-conservative formulations of the Euler equations at $y = 0.5$ location. This demonstrates that the PINNs framework yields a consistent shock-resolving capability irrespective of the chosen form of the governing equations for steady case. Table 7 quantitatively compares the L_1 error in pressure and the mass defect across the different methods. However, this localized accuracy comes at the cost of strict mass conservation; the non-conservative PINNs exhibits a mass defect of 3.39×10^{-3}, which is nearly an order of magnitude higher than that of the conservative PINNs (3.62×10^{-4}). As expected, the FVM ensures perfect mass conservation.

Table 7: Error and Mass Conservation for supersonic flow over wedge

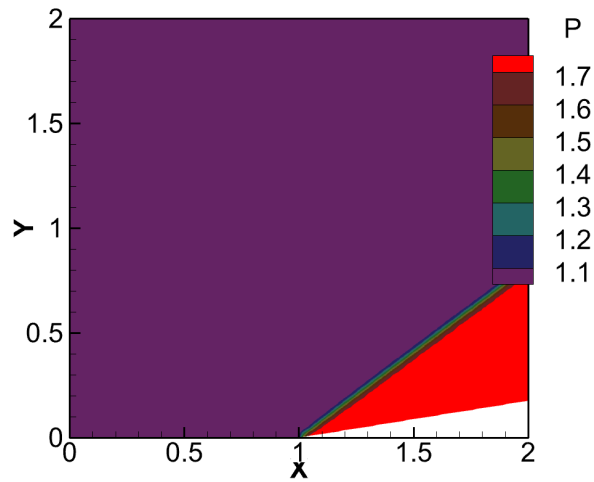
Method	L_1 Error in p	Mass Defect
PINNs (Conservative)	0.0202	3.62×10^{-4}
PINNs (Non-conservative)	0.0849	3.39×10^{-3}
Conservative FVM	0.01150	5.84×10^{-12}
Non-conservative-μ	0.11929	6.29×10^{-2}

6 Conclusion

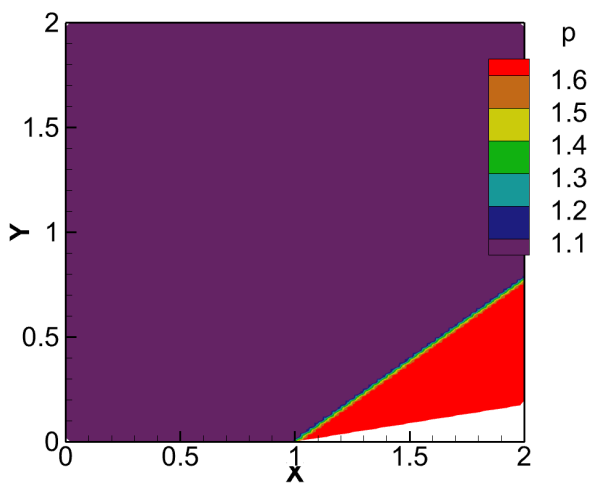
This study provides a comprehensive evaluation of conservative and non-conservative governing equations, tracing their performance from classical numerical solvers to physics-informed neural networks (PINNs) frameworks. By investigating the transition from scalar conservation laws to complex systems, we have identified the mechanisms that govern shock-capturing accuracy in neural-based solvers. Our analysis of the Euler equations reveals that standard non-conservative partial differential equations (PDEs) consistently produce incorrect shock speeds when solved using both PINNs and traditional numerical methods—even when augmented with adaptive or artificial viscosity. These inaccuracies in shock propagation are not merely a result of discretization errors or insufficient resolution; rather, they stem from a fundamental structural deficiency: the inherent inability of standard non-conservative formulations to satisfy the Rankine-Hugoniot conditions across discontinuities. In steady-state problems, the solution is independent of shock propagation speed. As a result, even non-conservative formulations can predict correct shock locations. However, such formulations may introduce non-physical mass generation or loss due to the lack of discrete conservation. In the context of PINNs, this issue can be slightly mitigated by incorporating additional mass-conservation constraints into the loss function.



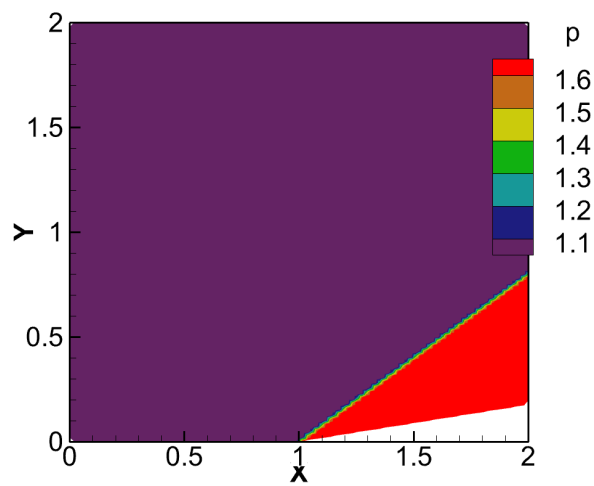
(a) Exact solution



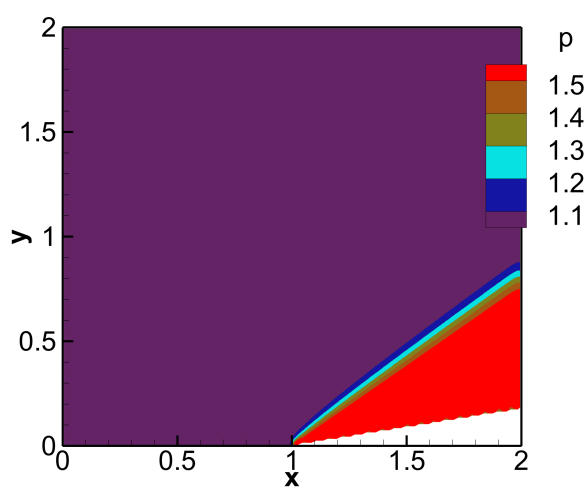
(b) Conservative numerical method



(c) PINNs on Conservative equations



(d) PINNs on non-conservative equations



(e) Non-Conservative-μ numerical method

Figure 10: Pressure solution of supersonic flow over wedge

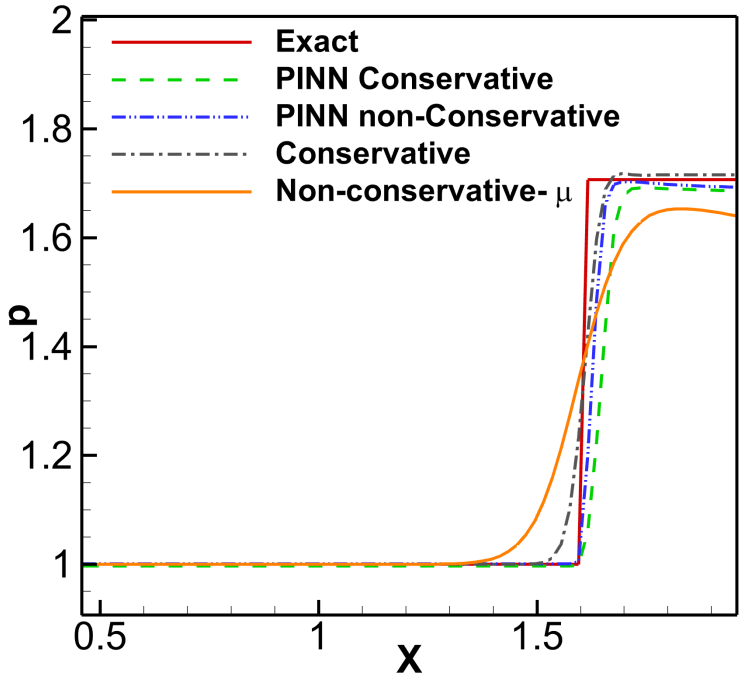


Figure 11: Pressure at y=0.5 in supersonic flow over wedge

However, this study demonstrates that this limitation can be successfully overcome through the integration of a **Path-Integral (Path-Conservative) formulation** based on the Dal Maso-LeFloch-Murat (DLM) theory. By incorporating a path-consistent loss function that integrates the system Jacobian along a defined state-space path, we found that:

- **Recovery of Physical Consistency:** The Path-Integral approach allows both numerical methods and PINNs to recover the mathematically correct shock speeds, effectively bridging the gap between primitive-variable formulations and physical conservation laws.

- **Resolution of Regularization Discrepancies:** The shock-speed drift observed in standard primitive-variable formulations is rooted in non-vanishing source terms introduced by viscous regularization. The path-integral constraint effectively nullifies these errors by ensuring the localized gradient satisfies the required jump manifold.

- **Neural Network Versatility:** PINNs-AWV demonstrates a unique capability to enforce these path-integral constraints globally. While conservative formulations remain the most straightforward for transient systems, the path-integral framework provides a robust alternative for applications where primitive variables are preferred or conservative forms are analytically complex.

Overall, this study establishes that conservativeness is not merely a property of the governing equations, but a hierarchical requirement that must be enforced at the mathematical, numerical, and physical levels. The proposed framework offers a principled pathway for designing physically consistent machine learning solvers for high-speed compressible flows.

Data and Code Availability

The source code for this project is publicly available on GitHub at `https://github.com/AGN000/Conservative-And-Non-conservative-PINNs-paper`.

References

[1] A Arun Govind Neelan, Manoj T Nair, and Raimund Bürger. Three-level order-adaptive weighted essentially non-oscillatory schemes. *Results in Applied Mathematics*, 12:100217, 2021.

[2] A Arun Govind Neelan, Raimund Bürger, Manoj T Nair, and Samala Rathan. Higher-order conservative discretizations on arbitrarily varying non-uniform grids. *Computational and Applied Mathematics*, 44(1):27, 2025.

[3] Eleuterio F. Toro. *Riemann Solvers and Numerical Methods for Fluid Dynamics: A Practical Introduction*. Springer, Berlin, Heidelberg, 3rd edition, 2009.

[4] Evgeniy Shapiro and Dimitris Drikakis. Non-conservative and conservative formulations of characteristics-based numerical reconstructions for incompressible flows. *International journal for numerical methods in engineering*, 66(9):1466–1482, 2006.

[5] Hervé Guillard and Claude Viozat. On the behavior of upwind schemes in the low mach number limit. *Computers & Fluids*, 28(1):63–86, 1999.

[6] Eli Turkel. Preconditioning techniques in computational fluid dynamics. *Annual Review of Fluid Mechanics*, 31(1):385–416, 1999.

[7] JG Verwer. Conservative and non. conservative schemes for the solution of the nonlinear schrodinger equation. *IMA journal of Numerical analysis*, 6:25–42, 1986.

[8] Yujie Li, Jiahui Zhang, Yinhua Xia, and Yan Xu. A path-conservative ader discontinuous galerkin method for non-conservative hyperbolic systems: Applications to shallow water equations. *Mathematics*, 12(16):2601, 2024.

[9] R. J. LeVeque. *Finite Volume Methods for Hyperbolic Problems*. Cambridge University Press, Cambridge, UK, 2002.

[10] M. R. Baer and J. W. Nunziato. A two-phase mixture theory for the deflagration-to-detonation transition (ddt) in reactive granular materials. *International Journal of Multiphase Flow*, 12(6):861–889, 1986.

[11] S. K. Godunov and E. I. Romenskii. *Elements of Continuum Mechanics and Conservation Laws*. Springer, Springer, 2003.

[12] F. Kemm, D. Kröner, C. D. Munz, T. Schnitzer, and M. Wesenberg. Hyperbolic divergence cleaning for the mhd equations. *Journal of Computational Physics*, 175(2):645–673, 2002.

[13] A. Aw and M. Rascle. Resurrection of "second order" models of traffic flow. *SIAM Journal on Applied Mathematics*, 60(3):916–938, 2000.

[14] Michael A. Celia, Elias T. Bouloutas, and Ronald L. Zarba. *A general mass-conservative numerical solution for the unsaturated flow equation*, volume 26. Wiley, AGU, 1990.

[15] Roger I. Tanner and Ken Walters. *Rheology: An Historical Perspective*. Elsevier, 2013.

[16] Herbert Goldstein, Charles Poole, and John Safko. *Classical Mechanics*. University of Toronto, 3rd edition.

[17] Mariano Castro and Carlos Parés. Well-balanced high-order finite volume methods for systems of balance laws. *Journal of Scientific Computing*, 27(1-3):107–131, 2006.

[18] Gianni Dal Maso, Philippe G LeFloch, and François Murat. Definition and weak stability of nonconservative products. *Journal de Mathématiques Pures et Appliquées*, 74(6):483–548, 1995.

[19] Ulrik Skre Fjordholm and Siddhartha Mishra. Accurate numerical discretizations of non-conservative hyperbolic systems. *ESAIM: Mathematical Modelling and Numerical Analysis*, 46(1):187–206, 2012.

[20] Christophe Berthon and Frédéric Coquel. Augmented roe schemes for nonconservative hyperbolic systems. *SIAM Journal on Scientific Computing*, 29(6):2370–2397, 2007.

[21] J.M. Greenberg and A.Y. LeRoux. Well-balanced schemes for conservation laws with source terms. *SIAM Journal on Numerical Analysis*, 33(1):1–16, 1996.

[22] Ami Harten. Upstream differencing and godunov-type schemes for hyperbolic conservation laws. *SIAM Review*, 25(1):35–61, 1983.

[23] Marsha J. Berger and Phillip Colella. Local adaptive mesh refinement for shock hydrodynamics. *Journal of Computational Physics*, 82(1):64–84, 1989.

[24] Ronald Fedkiw, X-D Liu, and Stanley Osher. A general technique for eliminating spurious oscillations in conservative schemes for multiphase and multispecies euler equations. *International Journal of Nonlinear Sciences and Numerical Simulation*, 3(2):99–106, 2002.

[25] Deep Ray. A data-driven approach to predict artificial viscosity in high-order solvers. In *2022 Spring Central Sectional Meeting*. AMS, 2021.

[26] Léo Bois, Emmanuel Franck, Laurent Navoret, and Vincent Vigon. An optimal control deep learning method to design artificial viscosities for discontinuous galerkin schemes. *arXiv preprint arXiv:2309.11795*, 2023.

[27] Emilio Jose Rocha Coutinho, Marcelo Dall'Aqua, Levi McClenny, Ming Zhong, Ulisses Braga-Neto, and Eduardo Gildin. Physics-informed neural networks with adaptive localized artificial viscosity. *Journal of Computational Physics*, 489:112265, 2023.

[28] Arun Govind Neelan, G Sai Krishna, and Vinoth Paramanantham. Physics-informed neural networks and higher-order high-resolution methods for resolving discontinuities and shocks: A comprehensive study. *Journal of Computational Science*, 83:102466, 2024.

[29] Peter D. Lax. Weak solutions of nonlinear hyperbolic equations and their numerical computation. *Communications on Pure and Applied Mathematics*, 7(1):159–193, 1954.

[30] Ami Harten. High resolution schemes for hyperbolic conservation laws. *Journal of Computational Physics*, 49(3):357–393, 1983.

[31] Thomas Y. Hou and Philippe G. LeFloch. Why nonconservative schemes converge to wrong shock solutions: Error analysis. *Mathematics of Computation*, 62(206):497–530, 1994.

[32] Randall J LeVeque. *Numerical Methods for Conservation Laws*. Birkhäuser Basel, 1992.

[33] Peter D Lax and Burton Wendroff. Systems of conservation laws. *Communications on Pure and Applied Mathematics*, 13(2):217–237, 1960.

[34] Carlos Parés. Numerical methods for nonconservative hyperbolic systems: a theoretical framework. *SIAM Journal on Numerical Analysis*, 44(1):300–321, 2006.

[35] John Von Neumann and Robert D Richtmyer. A method for the numerical calculation of hydrodynamic shocks. *Journal of Applied Physics*, 21(3):232–237, 1950.

[36] Per-Olof Persson and Jaime Peraire. Sub-cell shock capturing for discontinuous galerkin methods. In *44th AIAA Aerospace Sciences Meeting and Exhibit*, page 112, 2006.

[37] Eitan Tadmor. Entropy stability theory for difference approximations of nonlinear conservation laws and related time-dependent problems. *Acta Numerica*, 12:451–512, 2003.

[38] Manuel J Castro, Ulrik S Fjordholm, Siddhartha Mishra, and Carlos Parés. Entropy conservative and entropy stable schemes for nonconservative hyperbolic systems. *SIAM Journal on Numerical Analysis*, 51(3):1371–1391, 2013.

[39] Vikas Dwivedi, Nishant Parashar, and Balaji Srinivasan. Distributed physics informed neural network for data-efficient solution to partial differential equations. *arXiv preprint arXiv:1907.08967*, 2019.

[40] Khemraj Shukla, Ameya D Jagtap, and George Em Karniadakis. Parallel physics-informed neural networks via domain decomposition. *Journal of Computational Physics*, 447:110683, 2021.

[41] Ameya D Jagtap, Ehsan Kharazmi, and George Em Karniadakis. Conservative physics-informed neural networks on discrete domains for conservation laws: Applications to forward and inverse problems. *Computer Methods in Applied Mechanics and Engineering*, 365:113028, 2020.

[42] Ameya D Jagtap and George Em Karniadakis. Extended physics-informed neural networks (xpinns): A generalized space-time domain decomposition based deep learning framework for nonlinear partial differential equations. *Communications in Computational Physics*, 28(5), 2020.

[43] Ehsan Kharazmi, Zhongqiang Zhang, and George Em Karniadakis. hp-vpinns: Variational physics-informed neural networks with domain decomposition. *Computer Methods in Applied Mechanics and Engineering*, 374:113547, 2021.

[44] Stefano Berrone and Moreno Pintore. Meshfree variational-physics-informed neural networks (mf-vpinn): An adaptive training strategy. *Algorithms*, 17(9):415, 2024.

[45] Manvendra P Rajvanshi and David I Ketcheson. Integral pinns for hyperbolic conservation laws. In *ICLR 2024 Workshop on AI4DifferentialEquations In Science*, 2024.

[46] Ben Moseley, Andrew Markham, and Tarje Nissen-Meyer. Finite basis physics-informed neural networks (fbpinns): a scalable domain decomposition approach for solving differential equations. *Advances in Computational Mathematics*, 49(4):62, 2023.

[47] Zixue Xiang, Wei Peng, Xu Liu, and Wen Yao. Self-adaptive loss balanced physics-informed neural networks. *Neurocomputing*, 496:11–34, 2022.

[48] Dhruv Patel, Deep Ray, Michael RA Abdelmalik, Thomas JR Hughes, and Assad A Oberai. Variationally mimetic operator networks. *Computer Methods in Applied Mechanics and Engineering*, 419:116536, 2024.

[49] Lu Lu, Pengzhan Jin, Guofei Pang, Zhongqiang Zhang, and George Em Karniadakis. Learning nonlinear operators via deeponet based on the universal approximation theorem of operators. *Nature machine intelligence*, 3(3):218–229, 2021.

[50] Zongyi Li, Hongkai Zheng, Nikola Kovachki, David Jin, Haoxuan Chen, Burigede Liu, Kamyar Azizzadenesheli, and Anima Anandkumar. Physics-informed neural operator for learning partial differential equations. *ACM/JMS Journal of Data Science*, 1(3):1–27, 2024.

[51] Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Fourier neural operator for parametric partial differential equations. *arXiv preprint arXiv:2010.08895*, 2020.

[52] Zhiping Mao, Ameya D Jagtap, and George E Karniadakis. Physics-informed neural networks for high-speed flows. *Journal of Computational Physics*, 412:109439, 2020.

[53] Renzheng Wang, Yang Wang, and George E Karniadakis. Learning shock waves in multiphase flows using physics-informed neural networks. *Physics of Fluids*, 33(8):087114, 2021.

[54] Simon Wassing, Stefan Langer, and Philipp Bekemeyer. Physics-informed neural networks for transonic flows around an airfoil. *arXiv preprint arXiv:2408.17364*, 2024.

[55] Hong Liang, Zilong Song, Chong Zhao, and Xin Bian. Continuous and discontinuous compressible flows in a converging–diverging channel solved by physics-informed neural networks without exogenous data. *Scientific Reports*, 14(1):3822, 2024.

[56] Prashant Kumar and Rajesh Ranjan. A robust data-free physics-informed neural network for compressible flows with shocks. *Computers & Fluids*, 308:106975, 2026.

[57] Jongmok Lee, Seungmin Shin, Taewan Kim, Bumsoo Park, Ho Choi, Anna Lee, Minseok Choi, and Seungchul Lee. Physics informed neural networks for fluid flow analysis with repetitive parameter initialization. *Scientific Reports*, 15(1):1–16, 2025.

[58] Luciano Drozda, Pavanakumar Mohanamuraly, Lionel Cheng, Corentin Lapeyre, Guillaume Daviller, Yuval Realpe, Amir Adler, Gabriel Staffelbach, and Thierry Poinsot. Learning an optimised stable taylor-galerkin convection scheme based on a local spectral model for the numerical error dynamics. *Journal of Computational Physics*, 493:112430, 2023.

[59] Ritesh Kumar Dubey, Anupam Gupta, Vikas Kumar Jayswal, and Prashant Kumar Pandey. Learning numerical viscosity using artificial neural regression network. In *Computational Sciences-Modelling, Computing and Soft Computing: First International Conference, CSMCS 2020, Kozhikode, Kerala, India, September 10-12, 2020, Revised Selected Papers 1*, pages 42–55. Springer, 2021.

[60] David I. Ketcheson, Kyle T. Mandli, Aron J. Ahmadia, Amal Alghamdi, Manuel Quezada de Luna, Matteo Parsani, Matthew G. Knepley, and Matthew Emmett. PyClaw: Accessible, Extensible, Scalable Tools for Wave Propagation Problems. *SIAM Journal on Scientific Computing*, 34(4):C210–C231, November 2012.

[61] Randall J. LeVeque and David L. George. High-resolution finite volume methods for the shallow water equations with bathymetry and dry states. In *Advances in Coastal and Ocean Engineering*, volume 10, pages 43–73. World Scientific, 2008.

[62] A Arun Govind Neelan, R Jishnu Chandran, Manuel A Diaz, and Raimund Bürger. An efficient three-level weighted essentially non-oscillatory scheme for hyperbolic equations. *Computational and Applied Mathematics*, 42(2):70, 2023.

[63] Arun Govind Neelan and Manoj T Nair. Higher-order slope limiters for euler equation. *Journal of Applied and Computational Mechanics*, 8(3):904–917, 2022.

[64] A Arun Govind Neelan and Manoj T Nair. Hyperbolic runge–kutta method using evolutionary algorithm. *Journal of Computational and Nonlinear Dynamics*, 13(10):101003, 2018.

[65] María Luz Muñoz-Ruiz and Carlos Parés. Godunov method for nonconservative hyperbolic systems. *ESAIM: Mathematical Modelling and Numerical Analysis*, 41(1):169–185, 2007.

[66] Rémi Abgrall and Smadar Karni. A comment on the computation of non-conservative products. *Journal of Computational Physics*, 229(8):2759–2763, 2010.